Physics of Fluids

RESEARCH ARTICLE | JANUARY 02 2025

A physics-informed neural network for turbulent wake simulations behind wind turbines

Azhar Gafoor CTP ; Sumanth Kumar Boya ; Rishi Jinka ; Abhineet Gupta ; Ankit Tyagi ; Suranjan Sarkar ; Deepak N. Subramani

Check for updates

Physics of Fluids 37, 015110 (2025)
https://doi.org/10.1063/5.0245113

View Online Export Citation

Articles You May Be Interested In

Influence of atmospheric thermal stability on wake meandering of a large-scale wind turbine

Physics of Fluids (November 2025)

Multi-turbine turbulent wake simulation using physics-informed neural networks

Physics of Fluids (May 2026)

Flow in a large wind field with multiple actuators in the presence of constant vorticity

Physics of Fluids (October 2022)

12 September 2026 13:02:35

AIP Publishing

Physics of Fluids

Special Topics Open for Submissions

Learn More

AIP Publishing

Physics of Fluids ARTICLE pubs.aip.org/aip/pof

A physics-informed neural network for turbulent wake simulations behind wind turbines

Cite as: Phys. Fluids **37**, 015110 (2025); doi: 10.1063/5.0245113
Submitted: 23 October 2024 · Accepted: 3 December 2024 ·
Published Online: 2 January 2025

View Online Export Citation CrossMark

Azhar Gafoor CTP,[1] Sumanth Kumar Boya,[1] Rishi Jinka,[1] Abhineet Gupta,[2] Ankit Tyagi,[2] Suranjan Sarkar,[2] and Deepak N. Subramani[1,a)]

AFFILIATIONS

[1]Department of Computational and Data Sciences, Indian Institute of Science, Bangalore, Karnataka 560012, India
[2]Shell India Markets Pvt. Ltd., Bangalore, Karnataka 560103, India

[a)]Author to whom correspondence should be addressed: deepakns@iisc.ac.in

ABSTRACT

Fast simulations of wind turbine wakes are crucial during the design phase of optimal wind farm layouts. Wind turbine wakes affect the performance of downstream turbines. Physics-informed neural networks (PINNs), a deep learning approach to simulate dynamical systems governed by partial differential equations, are gaining traction in computational fluid dynamics due to their fast inference capability. We developed a PINN model using the 2-equation $k-\varepsilon$ model and the actuator disk method to simulate the wakes behind the wind turbines. Crucially, training of the developed PINN model does not rely on high-fidelity simulation data, thus reducing the end-to-end training time by saving simulation data generation time. We tested the model against traditional solvers and field data to simulate the turbulent wake behind the HOLEC WPS 30/3 Wind Turbine from Sexbierum and a three-blade 630-kW Nibe-B wind turbine. Detailed computational studies are completed to establish convergence properties with increasing sampling collocation points and the number of graphical processing units. A transfer learning strategy is introduced to accelerate training for new scenarios resulting in a 5× speedup. Our results establish the efficacy of the PINN model in simulating turbulent flows. Compared to field data, our PINN model and traditional Reynolds-averaged Navier–Stokes (RANS) numerical solvers, such as the shear stress transport k $-\omega$ and Reynolds stress model have similar errors, suggesting its utility as a replacement to these RANS solvers. The model architecture, trained weights, and code are available in https://github.com/quest-lab-iisc/PINN_WakeTurbulenceModel.

Published under an exclusive license by AIP Publishing. https://doi.org/10.1063/5.0245113

12 September 2026 13:02:35

I. INTRODUCTION

The simulation of turbulent flows plays a vital role in various scientific fields, such as renewable energy, astrophysics, geology, and weather forecasting. An estimate of the turbulent wake behind wind turbines is crucial for several applications, such as new turbine design, estimation of power output under different wind conditions, and optimization of the layout of wind farms. The total power output of a wind farm depends on the loss of wind speed in the wake of individual turbines. Thus, simulating the flow accurately behind one turbine is important. Furthermore, in low-resource economies such as India, Venezuela, etc., single-turbine power plants are common.[1–3] The goal of the present paper is to develop a neural network model as a fast surrogate for simulating the wake behind one wind turbine that is subject to wind.

The turbulent wake flow is governed by nonlinear Navier–Stokes (NS) equations. Traditionally, these equations are solved numerically either directly (called direct numerical simulation, DNS[4]) or using a turbulence closure mechanism. Large eddy simulation (LES)[5] and Reynolds-averaged Navier–Stokes (RANS)[6] are the popular turbulence closure models used. Among the two, RANS strikes a balance between accuracy and computational efficiency, making it a preferred choice for complex industrial problems for the foreseeable future.[7–9] Traditionally, simulations are performed by solving these equations on a discretized spatiotemporal domain using numerical schemes, such as finite difference, finite volume, finite element, and spectral methods. All of these methods require heavy computation on many-core systems and do not have the option for fast inference on a new configuration without solving the partial differential equations (PDEs) from scratch again.

Artificial Intelligence (AI) systems that use neural networks trained on graphical processing units (GPUs) have gained popularity across various domains, such as text and image processing, and more recently for numerical solution of PDEs. The major advantage of an AI solution is that once the system is trained, inference on new

Published under an exclusive license by AIP Publishing

configurations is fast. However, unlike text and image applications, generating data to train neural networks is expensive for fluid simulations. Data generation requires running traditional numerical methods on many core systems over days and weeks.[10] Fortunately, a methodology known as physics-informed neural networks (PINN) solves the problem of data requirement by using a new loss function that is the weighted sum of the residual PDE (representing the physics of the problem) and the mean square error of the data from the boundary/initial conditions.[11] The innovation was to use automatic differentiation[12] through the neural network to calculate the residual PDE, eliminating the need for simulation data to inform the physics of the neural model during training. Another prevalent approach to train these neural models (e.g., DeepONet,[13] Fourier Neural Operator,[14] and Representation equivalent Neural Operators[15]) is to leverage data from traditional PDE solvers for training. Although both of these approaches have shown success in simulating certain nonlinear PDEs, their applicability to highly turbulent flows remains unproven. Specifically, the development of PINN models to simulate turbulent flows governed by Reynolds-averaged Navier–Stokes (RANS) equations using residual-driven training has not been done previously. The PINN model also avoids the complex discretization process in conventional numerical solvers.

The purpose of the present paper is to address the above-mentioned research gap by developing and demonstrating the ability of PINN models to model the turbulent wake behind wind turbines. Our motivation is to perform training without using high-fidelity simulation data from traditional solvers. We want to address key methodological questions such as (i) What is the relationship between training collocation sampling points and solution convergence? (ii) What are the best set of hyperparameters including the loss weighting factor? (iii) What is the impact of using multiple GPUs for training? and (iv) Is transfer learning via trained model checkpoints feasible to reduce training time for new situations?

In what follows, we first review the previous progress in deep neural model development for turbulent fluid simulation. Next, we describe the general form of partial differential equations that govern turbulent flow (Sec. II A), followed by a brief explanation of the architecture of the physics-informed neural network and the training strategies (Sec. II B). Section III shows the 3D turbulent wake simulation around a single wind turbine using our PINN model and comparison with computational fluid dynamics (CFD) solution and field data. The computational requirements and the applicability of transfer learning to improve computational efficiency are also discussed (Sec. IV). Finally, we discuss our findings and present conclusive remarks with future work (Sec. V).

A. Previous progress in deep neural models for turbulent flow simulation

We categorize the literature for turbulence modeling using neural networks (or machine learning in general) into two buckets: (i) machine learning (ML)-augmented CFD methods and (ii) neural models for flow simulation. In the second bucket, there are models trained from CFD simulation data and those that are trained with a target of PDE residual.

Within the first bucket, the approaches in ML-augmented CFD have mainly been to model the unknown Reynolds stress tensor in the RANS equation from mean velocity and pressure fields using experimental or high-fidelity data from numerical methods.[16–21] Various machine learning techniques and models such as the kernel regression method,[22,23] random forest-based approaches,[24–26] and tensor-basis neural network (TBNN)[27] have been implemented for the same. Studies have been completed on the convergence of such coupled ML-CFD computations.[28,29] Another approach in ML-augment CFD is to use CFD on one part of the domain, use those data to train an ML model, and use it to predict other parts of the domain.[30,31] One major drawback is that using only DNS data (and not experimental data) to model closure terms in RANS can result in poor estimates due to the inconsistency between DNS and RANS models.[32–34] In addition, overfitting can cause the resultant ML models to violate the governing physical laws.[35] Estimation of PDE parameters and the discovery of equations, posed as inverse problems, have been tackled using symbolic regression,[36,37] sparse regression, and other machine learning methods.[38–41] PDE-Net uses convolutions to approximate differential operations and learn PDEs from data.[42,43]

In the second bucket, the first track is developing neural models for flow simulation using simulation data from traditional CFD methods to develop reduced-order models of fluid flow dynamics.[44–46] A data-driven surrogate to high-fidelity numerical flow simulations was developed employing a deep convolutional neural network trained using digital images.[47] Neural operator models[48] trained purely from simulation data have been developed on the basis of their underlying principles. The most popular are Fourier neural operators (FNO) based on the Fourier transform,[14] graph neural operators based on graph kernel learning,[49] and convolutional neural operator (CNO) based on convolutional neural networks (CNN) with U-Net architecture.[50] The second track is developing neural models by training against PDE residuals that are calculated using automatic differentiation through the neural network. These models are called physics-informed neural networks (PINNs).[11,51–53] The use of automatic differentiation is not new in CFD and has previously been used to facilitate derivative computation in other applications outside of neural network training.[54–56]

Some papers have reported the use of PDE-residual-driven neural models for turbulence studies. A PINN architecture with RANS formulation has been proposed to calculate the missing PDE parameters as an inverse problem.[57] The zero-equation turbulence model was used to incorporate RANS with PINN models to solve the conjugate heat transfer problem in NVIDIA SimNet.[58] RANS was incorporated into the PINN models by directly predicting unknown terms in the RANS equations using data on the domain boundaries, including Reynolds-stress components, which are obtained from numerical simulations or experiments.[59] A PINNs model for solving RANS equations showed better prediction than traditional two-equation models for one-dimensional Rayleigh–Taylor turbulent mixing problems.[60] A PINN-based surrogate model for turbulent flow problems using a k–ε RANS formulation was developed and pre-trained using data from traditional solvers before introducing PDE-residual loss for training.[61] In another PINN model, sparse DNS data were used together with PDE-residual loss.[62] An improvement of data-driven TBNN with inductive bias informed by turbulence realizability constraints, called physics-informed TBNN, has also been proposed.[63]

All the methods listed so far used simulation data for training either fully (track one) or partially (track two). One major goal of the present paper is to develop a PINN model to simulate turbulent flows

12 September 2026 13:02:35

Published under an exclusive license by AIP Publishing

governed by steady-state RANS equations without using simulation data from traditional CFD solvers. This will reduce the training data time, as well as avoid using high performance computing resources for generating simulation data from traditional solvers so as to train the new PINN models.

II. PINN MODEL DEVELOPMENT

A. Reynolds averaged Navier Stokes turbulent flow equations

The Reynolds-averaged Navier–Stokes equations (RANS equations) are time-averaged equations used to model turbulent fluid flows. These equations are derived by decomposing the turbulent flow variables into mean (time-averaged) and fluctuating parts, followed by time averaging. This approach introduces a Reynolds-stress term τ_{ij} that is modeled using a two-equation eddy viscosity turbulence closure model called the $k-\varepsilon$ model. Specifically, the $k-\varepsilon$ two-equation model includes transport equations for turbulent kinetic energy (k) and turbulent dissipation rate (ε).[64] The incompressible steady RANS equations, with the transport equations for the turbulent kinetic energy (k) and the dissipation rate (ε) in the Cartesian coordinates, are given as follows:

$$\text{Continuity}: \quad \frac{\partial u_i}{\partial x_i} = 0, \tag{1a}$$

$$\text{Momentum}: \quad \frac{\partial u_i u_j}{\partial x_j} = -\frac{1}{\rho}\frac{\partial p}{\partial x_i} + \frac{\partial}{\partial x_j}\left[\nu\left(\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i}\right)\right] - \frac{\partial \tau_{ij}}{\partial x_j} + S_i, \tag{1b}$$

$$\text{Reynolds-stress}: \quad \tau_{ij} = -\nu_t \frac{1}{2}\left(\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i}\right), \tag{1c}$$

$$k\text{-transport}: \quad \frac{\partial k u_j}{\partial x_j} = \frac{\partial}{\partial x_j}\left[\left(\nu + \frac{\nu_t}{\sigma_k}\right)\frac{\partial k}{\partial x_j}\right] + P_k - \varepsilon, \tag{1d}$$

$$\varepsilon\text{-transport}: \quad \frac{\partial \varepsilon u_j}{\partial x_j} = \frac{\partial}{\partial x_j}\left[\left(\nu + \frac{\nu_t}{\sigma_\varepsilon}\right)\frac{\partial \varepsilon}{\partial x_j}\right] + C_{1\varepsilon}\frac{\varepsilon}{k}P_k - C_{2\varepsilon}\frac{\varepsilon^2}{k} + S_\varepsilon, \tag{1e}$$

$$\nu_t = C_\mu \frac{k^2}{\varepsilon}, \tag{1f}$$

$$P_k = \nu_t\left(\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i}\right)\frac{\partial u_i}{\partial x_j}, \tag{1g}$$

where x_i is the spatial coordinate, u_i is the time-averaged velocity vector [u, w for 2D in space (x, z) and u, v, w for 3D in space (x, y, z)], p is the mean pressure, ν is the kinematic viscosity, ν_t is the turbulent viscosity, P_k is the production of turbulent kinetic energy, S_i is the momentum source/sink term in the i direction, and S_ε is the source/sink term for dissipation rate. For the 3D turbulent wake simulation around a single wind turbine under neutral atmospheric conditions, the constant values are $C_\mu = 0.033$, $\sigma_k = 1.0$, $\sigma_\varepsilon = 1.3$, $C_{1\varepsilon} = 1.176$, and $C_{2\varepsilon} = 1.92$, determined by regression fits of observation data.[65]

B. PINN architecture

Let us denote each equation in the $k-\varepsilon$ model [Eq. (1)] using the set of generic nonlinear functions

$$f_i(\mathbf{u}, \mathbf{x}) = 0, \quad \mathbf{x} \in \Omega, \tag{2a}$$

$$\mathscr{B}(\mathbf{u}(\mathbf{x})) = b, \quad \mathbf{x} \in \partial\Omega, \tag{2b}$$

where i denotes different PDEs (continuity, momentum, k-transport and ε-transport equation), $\mathscr{B}$ are the boundary and/or initial condition operator, $\mathbf{x} \in \Omega$ is the coordinates in the spatial domain Ω, $\partial\Omega$ is the boundary of the domain, b is the boundary condition value and $\mathbf{u} \in \mathscr{U}$ is the solution field in a solution space $\mathscr{U}$ that satisfies the above PDE. The solution field $\mathbf{u}$ consists of the flow velocities (u, v and w), pressure (p), turbulent kinetic energy (k), and dissipation rate (ε). $\mathbf{x}$ denotes the coordinates of the spatial domain (x, y, z).

The PINN approach uses and trains a parameterized neural model $\mathscr{H}$ that approximates the functional map between the input domain Ω and the solution domain $\mathscr{U}$, i.e., $\mathscr{H} : \mathbf{x} \times \theta \mapsto \mathbf{u}$, where θ is the set of trainable weights and biases, such that differential equations with their boundary condition are satisfied. Figure 1 shows the schematic of a PINN model. A fully connected feed-forward neural network, enclosed in the blue box, provides the output $(\hat{u}, \hat{v}, \hat{w}, \hat{p}, \hat{k}, \hat{\varepsilon})$ for the given input variables (x, y, z). Thus, $\hat{\mathbf{u}}(\mathbf{x}; \theta)$ is the approximate solution of the PDEs under consideration. The partial differential terms in the governing equations (e.g., $\partial\hat{\mathbf{u}}/\partial\mathbf{x}$ or higher-order terms) are computed using automatic differentiation. To compute the boundary condition losses, in the case of the Dirichlet boundary condition, the approximated outputs are compared with the boundary data directly and the automatic differentiation process is bypassed. If Neumann conditions are specified at the boundary, the losses are computed with the help of automatic differentiation. The parameters (θ) of the neural network $\mathscr{H}$ are learned by minimizing a loss function $\mathscr{L}_{PINN}$ that comprises the PDE-residual loss and boundary data loss, i.e.,

$$\mathscr{L}_{PINN} = \alpha_b \mathscr{L}_{boundary} + \alpha_r \mathscr{L}_{residual}, \tag{3a}$$

$$\mathscr{L}_{boundary} = \frac{1}{N_{bp}}\left[\sum_{i=1}^{N_{bp}} (\mathscr{B}(\hat{\mathbf{u}}(\mathbf{x_i})) - b)^2\right], \tag{3b}$$

$$\mathscr{L}_{residual} = \frac{1}{N_c}\left[\sum_{i=1}^{N_c} \alpha_1 (f_{continuity})^2 + \alpha_2 (f_{momentum})^2 + \alpha_3 (f_{k-transport})^2 + \alpha_4 (f_{\varepsilon-transport})^2\right]. \tag{3c}$$

Here, α_b is the weighting coefficient for boundary losses, α_r is the weighting coefficient for residual losses, and α_{1-4} are the weighting coefficients assigned to the residuals of each governing PDE. A hyperparameter sweep is used to fix these coefficients (see Appendix B). N_c represents the total count of location points within the domain and N_{bp} denotes the total number of boundary points. The boundary loss term is determined using the mean squared error with the boundary data provided. The PDE-residual loss is calculated as the L2 norm of the residuals at training points calculated using automatic differentiation. The training process is dominated by the PDE-regularization. Therefore, training of PINN models can be seen as minimizing a combination of a *supervised loss* calculated from the boundary conditions and a *unsupervised loss* calculated from the residual of the PDEs by automatic differentiation.

12 September 2026 13:02:35

Published under an exclusive license by AIP Publishing

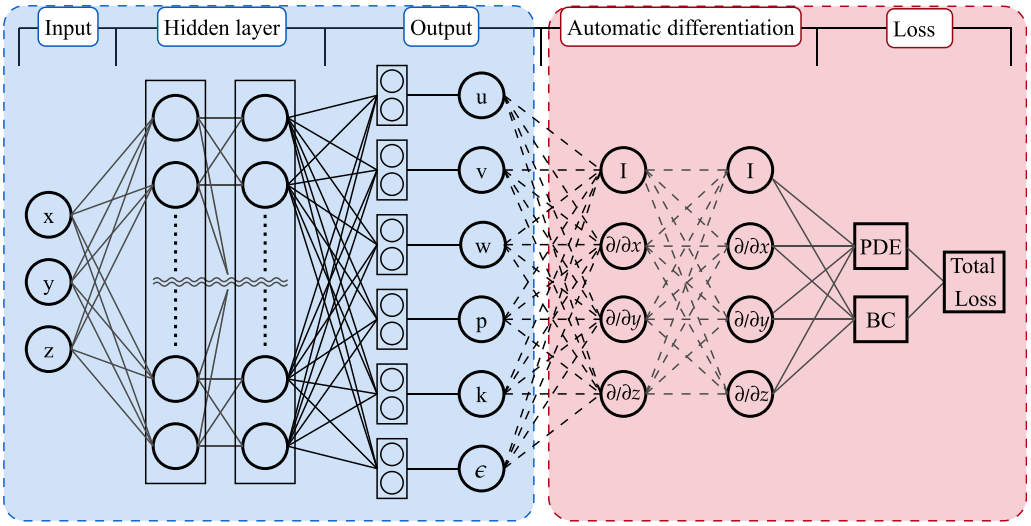


FIG. 1. Schematic *of a* PINN model for 3D RANS simulation. The PINN model is a fully connected feed-forward dense neural network shown in the blue enclosure. The loss calculation through automatic differentiation is visualized as dotted connections in the red enclosure. Two losses are calculated (i) partial differential equation (PDE) residual loss and (ii) boundary condition (BC) loss.

C. Wall functions

The standard model $k-\varepsilon$ is reliable for applications with a high Reynolds number that do not have flow separation and reattachment.[66] In such cases commonly encountered in industrial flow scenarios, the use of wall functions is preferred to achieve sufficiently precise simulations.[67] In conventional numerical methods, wall functions reduce computational cost by avoiding the need for fine grids. Placing the first node point sufficiently far in the overlap region of the turbulent boundary layer leads to a significant reduction in the number of cells near the wall or boundary.[68] Wall functions take advantage of the universality exhibited by the profiles near the wall at high Reynolds numbers. The logarithmic law of the wall is applied in the overlap region to obtain the mean flow quantities parallel to the wall. Moreover, the wall function approach allows the inclusion of empirical information in special cases, such as in rough wall conditions.[69]

We use wall functions in Eq. (3b) to ensure that our PINN model satisfies the flow profile near the wall. Unlike traditional CFD, the training of the PINN model depends on gradient descent-based optimization in a loss landscape formed by the combination of PDE losses and boundary losses. Hence, there is a possibility that the solution can get stuck in local minima or move toward a non-desired solution. Providing the mean flow quantities using the wall functions drives our PINN model toward the correct solution quickly. There are many applications where the focus is not on resolving the underlying physics in the boundary layer. In such cases, wall functions help to minimize the need for a high number of sample points.

D. Implementation

The PINN model above is implemented using the TensorFlow software package.[70] The architecture has two/three inputs (corresponding to 2D/3D applications), five/six outputs (2D/3D), four hidden layers that are shared by all outputs, and a separate additional layer for each output (Fig. 1). Our previous study showed that PINN models yield improved outcomes with a smaller number of hidden layers.[71] Training hyperparameters, including the number of neurons in each layer used to model a single wind turbine are chosen by repetitive experiments tracked with Weights and Biases[72] (Table I). Swish activation is used for all hidden layers,[73] ReLU activation is used for the output nodes corresponding to turbulent kinetic energy (k) and dissipation rate (ε) to avoid negative outputs, and no activation is used for the other output nodes. Adam optimizer is utilized to iteratively find the model parameters while training for a preset number of epochs with exponentially decaying learning rates starting with 0.0001. Adam is a first-order optimizer based on an adaptive gradient descent with momentum.[74] For parameter initialization, the Glorot uniform initializer is used.[75] Mini-Batch training is performed because of the large number of collocation points necessary for the 3D wake simulation. Training is carried out on a standalone workstation with two NVIDIA RTX A6000 GPUs. The code, the model architecture, and the trained weights have been released on GitHub with a Zenodo DOI.[76] The pseudo-code for training the developed RANS-PINN model is shown in Table II.

TABLE I. Hyperparameters used for single wind turbine modeling.

Neurons (hidden layers)	600
Neurons (output layer)	500
Learning rate	0.0001
Epochs	25 000
Optimizer	Adam
Trainable-parameters	2 890 200
Batches	12
Activation function	Swish + ReLU
Initializer	Glorot uniform

12 September 2026 13:02:35

Published under an exclusive license by AIP Publishing

TABLE II. Pseudo-code: RANS-PINN implementation.

$\mathscr{C} \leftarrow \mathscr{D} + \mathscr{B}, \mathscr{C}$ − Set of collocation points, $\mathscr{D}$ − Set of domain points, $\mathscr{B}$ − Set of boundary points

train_PINN: Randomly initialize weights $w_i = \{w_1, w_2, \ldots, w_n\}$
while *Epochs* < *Total_steps* **do** *forward_pass*
  input $\mathbf{x} = \{x, y, z\}$ in the input layer, $\mathbf{x} \in \mathscr{C}$
  output $= \{u, v, w, p, k, \epsilon\}$ in the output layer
  $Loss \leftarrow 0$
  Get PDE losses
  Use automatic differentiation to calculate the derivatives used in the PDEs.
  Solve Eq. (1).
    1. Calculate ν_t [Eq. (1f)].
    2. Calculate τ_{ij} and P_k [Eqs. (1c) and (1g)].
    3. *PDE_Loss* = residuals of continuity, momentum, k-transport and ϵ-transport equations [Eqs. (1a), (1b), (1d), and (1e)].
  $Loss+ = PDE_Loss$
  if x is in $\mathscr{B}$ **then** get Boundary Condition loss
    Dirichlet Boundary: Calculate the residuals using the boundary values.
    Neumann Boundary: Use Automatic Differentiation to calculate the derivatives and calculate the residuals using the boundary values.
    $Loss+ = BC_Loss$
  end if
  Use the *Loss* to calculate the gradients and update the weights.
end while

E. 2D model development

Before simulating the 3D turbulent wake, two benchmark test cases are simulated in the 2D spatial domain to verify our implementation and validate the consistency of our PINN model. The first test case is the simulation of a turbulent flow driven by a pressure gradient in a 2D channel. The solution obtained from our PINN model is compared with the solution of DNS and RANS simulations from traditional CFD tools. DNS simulation data of turbulent channel flow for friction Reynolds numbers of 180, 395, and 590 are used from the literature.[77,78] A simple in-house finite-volume code in MATLAB is used to obtain the RANS CFD data. The second test case simulates a streamwise homogeneous flow driven by velocity. For this case, the reference velocity and turbulent quantity profiles of the neutral stratified atmospheric boundary layer (ABL) flow from the literature are used.[79] Both these test cases are simulated using the 2D formulation of the governing PDEs Eq. (1) without any source term (Sec. II A). Using these flow scenarios, we also fine-tune the hyperparameters $\alpha_b, \alpha_r, \alpha_1, \alpha_2, \alpha_3$, and α_4 in Eq. (3). (See Appendix A for details on the 2D flow simulations and Appendix B for hyperparameter tuning.)

III. 3D TURBULENT FLOW BEHIND A WIND TURBINE

The PINN model is utilized to forecast the wake generated by a single wind turbine. This wake occurs due to the obstruction created by the wind turbine as it disrupts the incoming airflow. The turbine itself is represented using an actuator disk model[80,81] and is incorporated into the $k-\varepsilon$ RANS model as a source term (Sec. III A). The domain and experimental setup are discussed in Sec. III B. For the purpose of numerical simulations, the wake is computed for two separate standalone wind turbines, namely, the Sexbierum HOLEC WPS 30/3 Wind Turbine and the three-blade 630-kW Nibe-B wind turbine. The field and numerical solver datasets for the turbine cases are discussed in Sec. III C. Section III D shows a convergence study for the PINN model followed by the wake simulations in Sec. III E.

A. Actuator disk model source terms

The actuator disk model provides the source terms S_i and S_ε to be used in the RANS governing equations in the region around the wind turbine. The momentum source term (S_i) in Eq. (1b) imposes the thrust forces on the flow, which is distributed in the region representing the actuator disk. The equation for thrust exerted (T) and power extracted (P) by the disk are given by

$$T = \frac{1}{2}\rho C_T A_d u_0^2, \tag{4}$$

$$P = \frac{1}{2}\rho C_P A_d u_o^3, \tag{5}$$

where C_T is the thrust coefficient, C_P is the power coefficient, A_d is the area of the disk, and u_0 is the upstream velocity. C_T is a nondimensional number that describes the force exerted by the turbine in the axial direction on the incoming momentum of the flow, and C_P is defined as the fraction of the power extracted by the wind turbine relative to the energy available in the wind stream. For a wind turbine, the thrust and power coefficient are provided by the manufacturer. They are used to model the effect of all individual components of the wind turbine, such as blade shapes, structural properties, and generator and gearbox characteristics. In the present work, the thrust force represented by Eq. (4) is averaged by the turbine area and is normally distributed along the streamwise direction using the source term given as

12 September 2026 13:02:35

Published under an exclusive license by AIP Publishing

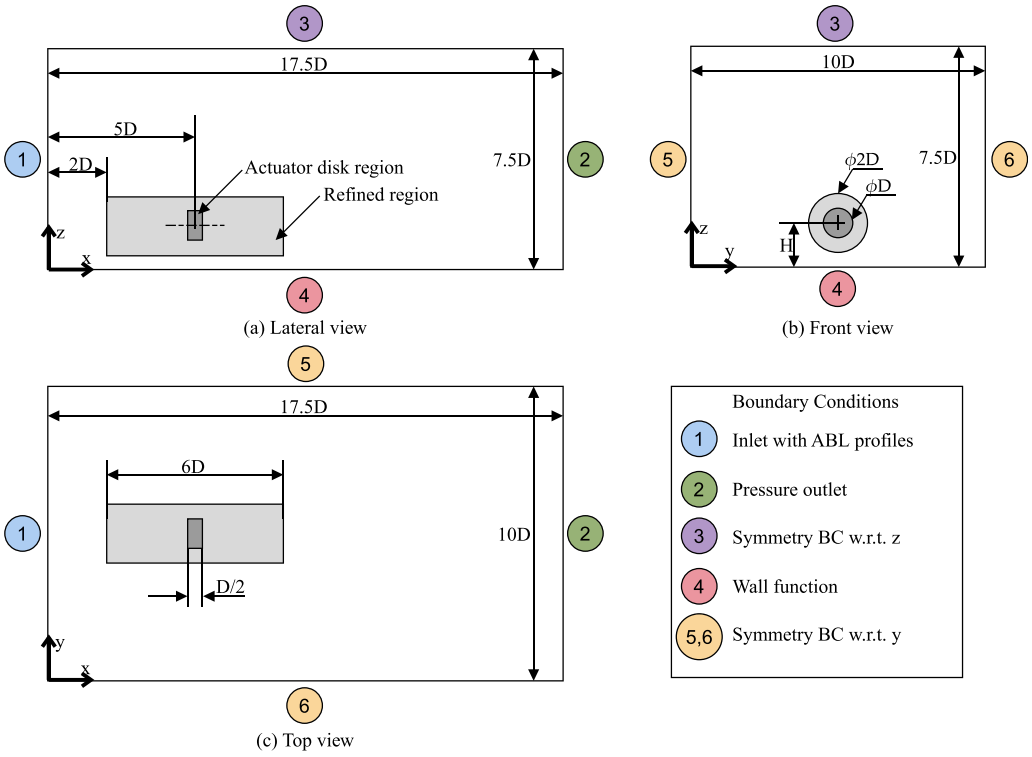


FIG. 2. Schematic of the single wind turbine modeling domain: (a) lateral view, (b) front view, and (c) top view. D is the diameter of the turbine and H is the hub height. The boundary conditions used on the domain surfaces are also shown.

$$S_i = -\frac{1}{2}\rho C_T u^2, \tag{6}$$

where u represents the far upstream velocity or the rotor averaged velocity based on which the thrust coefficient is correlated. The negative sign indicates that the thrust is applied to the incoming flow or S_i is a sink/negative source term. Although the standard $k-\varepsilon$ model for the RANS closure with the momentum source term can predict the presence of the disk, some flow variables are highly overestimated. Thus, an additional term that represents the rate of energy transfer from large-scale turbulence to small-scale turbulence controlled by the production range scale and the dissipation rate timescale is used in the $\varepsilon-$ transport equation [Eq. (1e)] to improve the simulation.[82,83] S_ε is defined as follows:

$$S_\varepsilon = C_{4\varepsilon}\frac{P_k^2}{\rho k}, \tag{7}$$

where $C_{4\varepsilon}$ is a constant whose value is set to 0.37 (from the reported experimental data[83]), P_k is the production of turbulent kinetic energy, and k is the turbulent kinetic energy. Note that the use of source terms is restricted only to the actuator disk region.

B. Domain and training

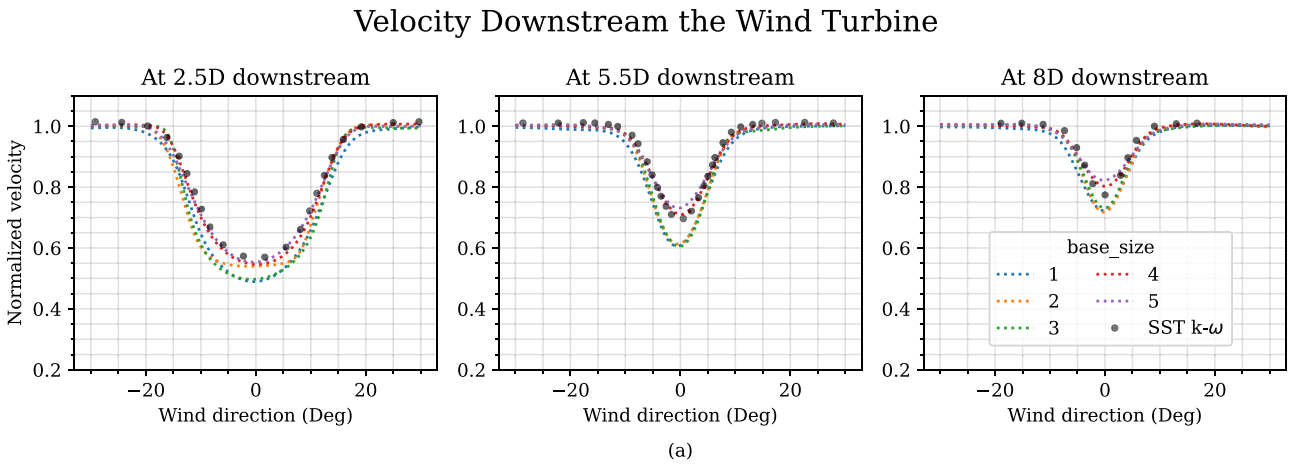
Figure 2 shows the schematic of the domain. During training of the PINN model, the boundary points at which boundary loss [Eq. (3b)] is imposed are sampled using a grid-based approach, and the domain points at which the PDE-residual loss [Eq. (3c)] is imposed are sampled using Hammersley sampling.[84,85] The region downstream of the wind turbine can be divided into two: near wake and far wake.[86] To obtain accurate wake simulations using the actuator disk model, the source terms are applied in a cylindrical region with a diameter equal to the turbine diameter (D) and a length of 0.25D upstream and downstream of the disk.[83] The highly heterogeneous near wake characterizes a complex flow distribution extending to a distance of 2 to 4D (rotor diameter) downstream. It is strongly influenced by the design of the rotor and the aerodynamic properties of the blades.[87,88] In this study, to precisely capture the behavior of the near-wake flow, the domain training points are increased in the cylindrical region with length = 6D and diameter = 2D, placed symmetrically to the disk (refined region in Fig. 2). This process is inspired by the mesh refinement used in traditional numerical solvers. In the far wake region, the turbine's influence is restricted to the thrust force and the turbulent kinetic energy. Thus, a smaller number of sampling points is sufficient compared to the near-wake region.

TABLE III. Wind turbine and flow specifications.

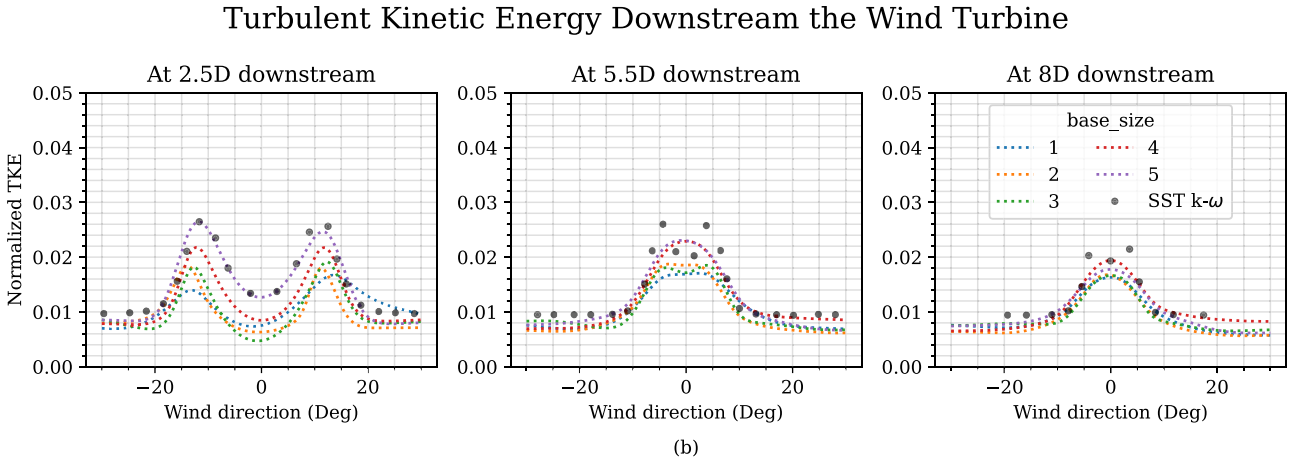
Wind turbine	D (m)	H (m)	U_{ref} (m/s)	TI_{ref} (%)	C_T
Sexbierum HOLEC WPS 30/3	30.1	35	10	10	0.75
Nibe-B wind turbine	40	45	8.5	10	0.82

12 September 2026 13:02:35

Published under an exclusive license by AIP Publishing



(a)



(b)

FIG. 3. Comparison of (a) velocity and (b) turbulent kinetic energy predicted by PINN model for a varying number of sample points with RANS-CFD simulation at 2.5D, 5.5D, and 8D downstream the Sexbierum HOLEC WPS 30/3 wind turbine.

Fully developed inlet profiles are specified at the inlet boundary for mean velocity [Eq. (8a)], turbulent kinetic energy [Eq. (8b)] and turbulent dissipation rate [Eq. (8c)] as follows:[79]

$$u = \frac{u_\tau}{\kappa} \ln\left(\frac{z + z_0}{z_0}\right), \tag{8a}$$

$$k = \frac{u_\tau^2}{\sqrt{C_\mu}}, \tag{8b}$$

$$\varepsilon = \frac{u_\tau^3}{\kappa(z + z_0)}, \tag{8c}$$

where $u_\tau = \sqrt{\frac{\tau_w}{\rho}}$ is the friction velocity (with τ_w is the wall shear stress, and ρ is the fluid density), $\kappa \approx 0.418$ is the von Kármán constant, z is the height co-ordinate, and z_0 is the aerodynamic roughness height. Inlet profiles [Eq. (8)] are analytical solutions of the momentum and $k - \varepsilon$ model equations.[89] At the outlet, a provided pressure boundary condition and zero gradients along the streamwise direction for other quantities are imposed. At the bottom wall/ground, wall functions consistent with the inlet profiles are used to specify Dirichlet conditions for velocity, turbulent kinetic energy, and dissipation rate. The top face and additional lateral faces in the 3D domain are provided with symmetry boundary conditions with a zero velocity normal to the face.

We perform simulations for two separate stand-alone wind turbines, the HOLEC WPS 30/3 Wind Turbine from Sexbierum, and a three-blade 630-kW Nibe-B wind turbine. The HOLEC WPS 30/3 WT

TABLE IV. Computational time for PINN model for single turbine wake simulation for different base sizes.

base_size	Time for 1000 epochs (min)
1	70
2	91
3	112
4	132
5	156

12 September 2026 13:02:35

Published under an exclusive license by AIP Publishing

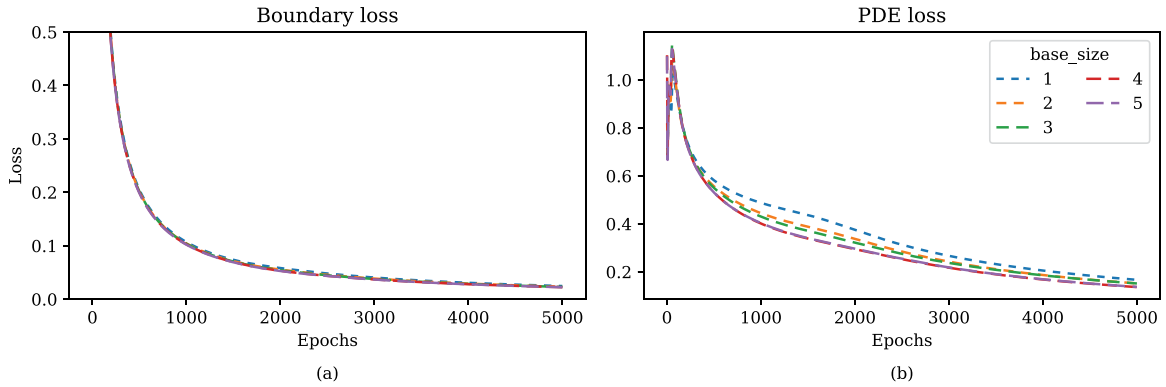


FIG. 4. (a) Boundary and (b) PDE loss of the PINN model for a varying number of sample points.

has a diameter D = 30.1 m and a hub height H = 35 m. The boundary conditions are set for a free stream wind speed of 10 m/s with a turbulence intensity of 10% at the hub height. A thrust coefficient of 0.75 corresponding to these conditions is also used. The Nibe wind turbine "B" has a diameter D = 40 m and a hub height H = 45 m. Here, a free stream wind speed of 8.5 m/s and turbulence intensity of 10% at the hub height were used with a corresponding thrust coefficient of 0.82. Table III summarizes the wind turbine and flow specifications. For each simulation, the PINN model is trained from scratch.

C. Data from field and conventional CFD

The validation of the PINN model with the turbulence closures and ABL inflow equations was performed using the field and numerical solver data sets for the standalone Sexbierum[90] and

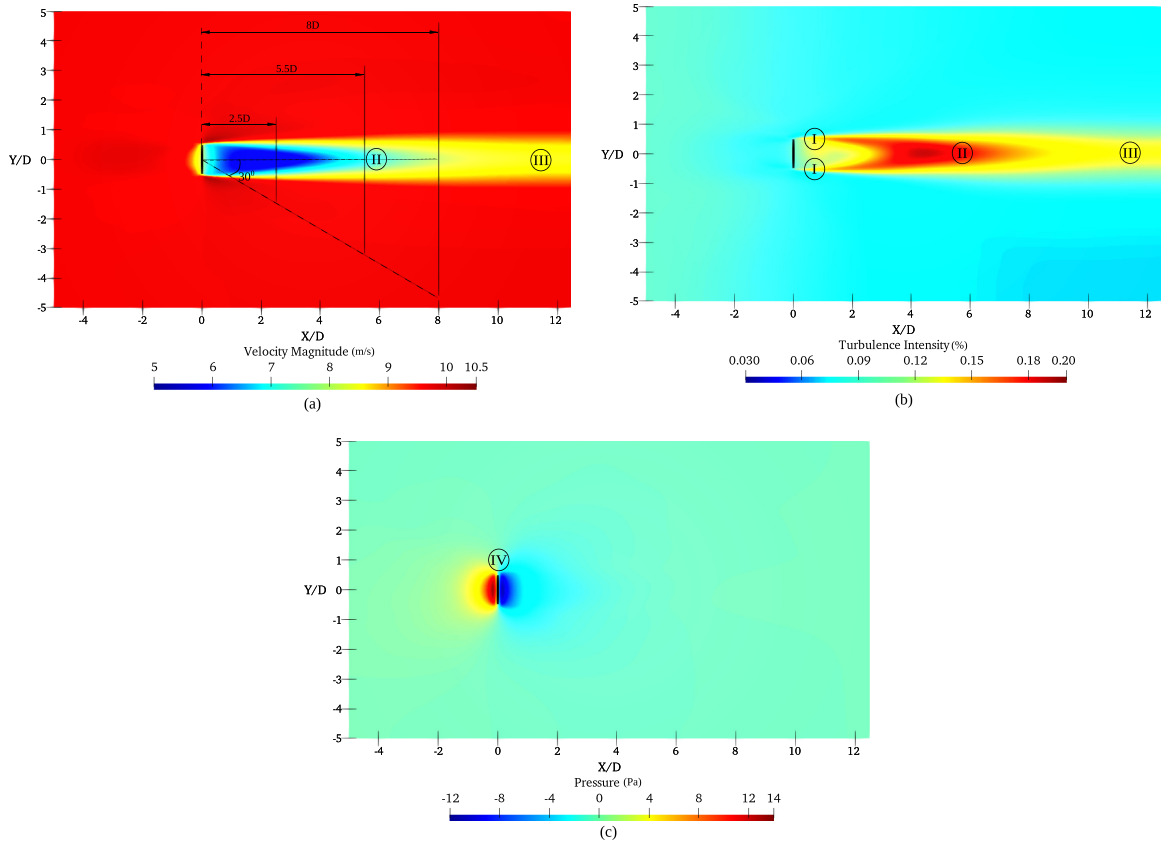


FIG. 5. Contours of (a) velocity, (b) turbulence intensity, and (c) pressure on a horizontal plane at hub height for the Sexbierum HOLEC WPS 30/3 wind turbine. Solid black line represents the position of the turbine. A quantitative comparison is made at a distance of 2.5D, 5.5D, and 8D downstream of the turbine.

12 September 2026 13:02:35

Published under an exclusive license by AIP Publishing

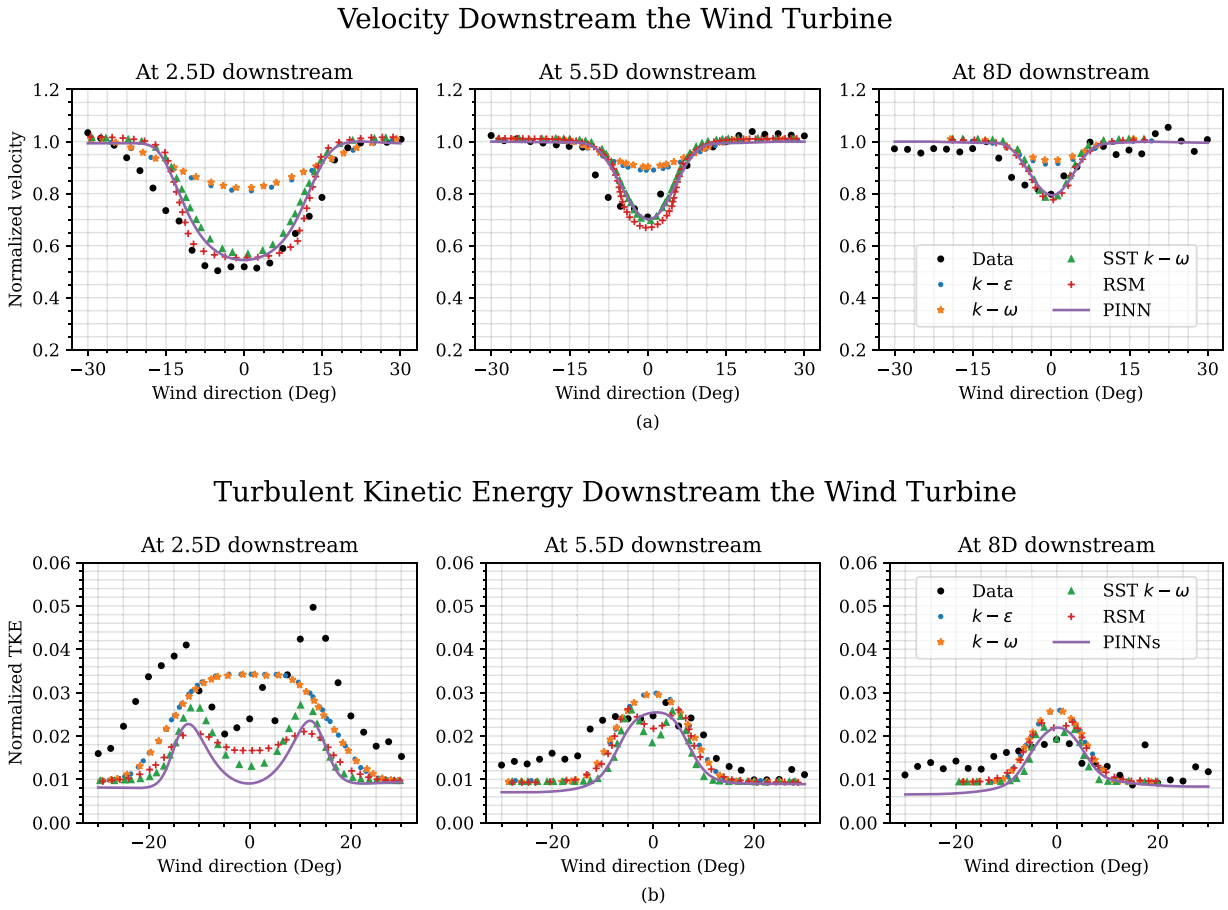


FIG. 6. Comparison of (a) velocity and (b) turbulent kinetic energy predicted by PINN model with data from experiment and numerical solvers at 2.5D, 5.5D, and 8D downstream the Sexbierum HOLEC WPS 30/3 wind turbine.

Nibe[91] wind turbines. The *single wake measurement dataset* from the Sexbierum turbine contains the normalized velocity and turbulent kinetic energy behind a single wind turbine at distances of 2.5, 5.5, and 8 turbine diameters. The data reported corresponding to the Nibe-B wind turbine operating alone consists of the measurements of normalized velocity and turbulence intensity behind the turbine at distances of 2.5, 4, and 7.5 turbine diameters. The numerical simulation using the finite volume method is performed using the Simcenter STAR-CCM+ software package.[92] Different turbulence models, namely standard $k-\varepsilon$,[64] standard $k-\omega$,[93] shear stress transport (SST) $k-\omega$,[94] and Reynolds stress model (RSM)[95,96] were implemented using STAR-CCM+. An appropriate mesh size is chosen by performing a mesh convergence analysis to reduce spatial discretization errors in the CFD simulations and to guarantee a mesh-independent solution (Appendix C).

D. Convergence of PINN model with number of sampling points

In the PINN model, the accuracy of the training depends on the number of randomly sampled points within the domain at which the PDE-residual loss is imposed. Here, a study is done to analyze the dependency on the number of sampling points and the convergence of the PINN model. The number of sampling points is chosen such that *base_size* $\times 10^4$ points are used for the whole domain, *base_size* $\times 10^3$ points are used for the refined region, and *base_size* $\times 10^3$ are used for the actuator disk region. We vary the base size from 1 to 5 to study convergence. Figure 3 shows the simulation of the wake behind a single wind turbine of the Sexbierum wind farm for different base sizes. The SST $k-\omega$ numerical simulation using the Simcenter STAR-CCM+ at downstream distances of 2.5, 5.5, and 8 turbine diameters are also provided to show how the PINN model is able to capture the variations in different flow parameters. The SST k–ω model is an effective turbulence model for the wind turbine simulations.[97] All simulations capture the trends in velocity and turbulent kinetic energy (TKE) variation in the near-downstream (2.5D) well. However, moving further downstream, the velocity recovery is less in simulations with base sizes of 1, 2, and 3. For base sizes 4 and above, the simulation shows good agreement with the experimental data even in the far downstream. The TKE also shows a similar trend. The increase in base size from 4 to 5 gave a barely noticeable improvement in the simulations, especially considering the velocity plots. A detectable change in the

12 September 2026 13:02:35

Published under an exclusive license by AIP Publishing

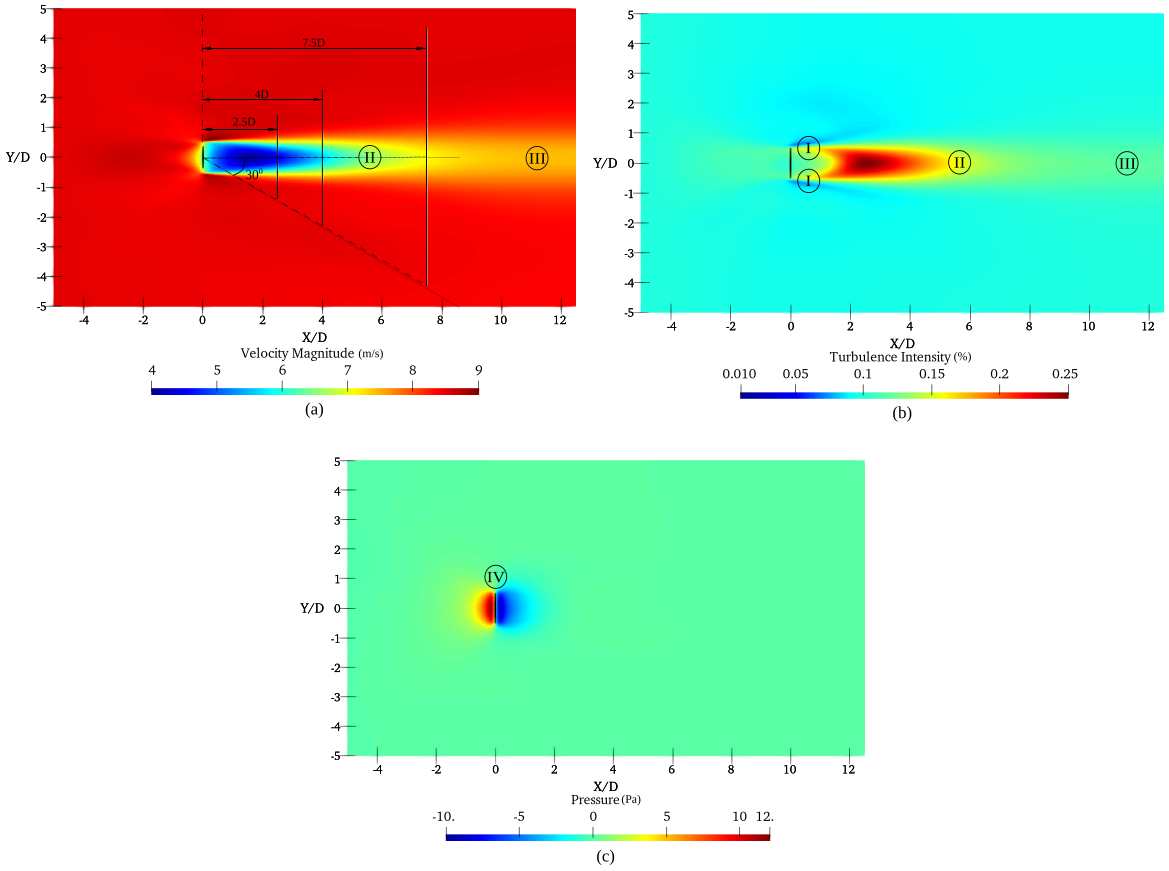


FIG. 7. Contours of (a) velocity, (b) turbulence intensity, and (c) pressure on a horizontal plane at hub height for the Nibe-B wind turbine. Solid black line represents the position of the turbine. A quantitative comparison is made at a distance of 2.5D, 4D, and 7.5D downstream of the turbine.

simulation is observed only in the near downstream TKE. However, this gain is shadowed by the increasing computational cost with the base size when training the model for higher epochs. Table IV lists the computational time for different base sizes. The simulation convergence does not exhibit a proportional relationship with the increase in the base size. Meanwhile, mesh sensitivity analysis of the numerical simulation using the finite volume method shows that as the mesh size is reduced, the simulation is better (see Appendix C). During training, it is observed that PDE physics is learned better (lower residual error of PDE) with an increasing number of samples as expected (Fig. 4). The loss curves also show no improvement with an increase in the base size from 4 to 5.

E. Wake simulations from the trained PINN model

Figure 5 shows the contour plot of the RANS-PINN turbulent flow simulations (velocity, turbulence intensity and pressure), and Fig. 6 shows the normalized velocity and Turbulent Kinetic Energy profiles at three locations (2.5D, 5.5D, and 8D) downstream of the Sexbierum turbine. In the latter figure, data from the field and conventional CFD simulations are shown for comparison. Figures 7 and 8 show the same for the Nibe turbine. Root-mean square errors (RMSE) were calculated for the simulation results from different turbulence models and the PINN model by comparing with the experimental data. Also, the PINN model simulation is compared with the numerical solvers to assess the performance of PINN model. The RMSE for normalized velocity and TKE for the Sexbierum wind turbine are reported in Table V, and the RMSE for normalized velocity and turbulence intensity for Nibe wind turbine are reported in Table VI. The simulations from numerical solvers and PINN model are further evaluated by plotting the absolute difference between the simulations and the experimental data[90,91] (Figs. 9 and 10). The figures show that the PINN simulation closely follows the variation and values exhibited by the SST k–ω model compared to other numerical models. Also, a study on different turbulence models concludes that the SST k–ω model is an effective turbulence model for wind turbine simulations.[97] Hence, an absolute difference plot (Figs. 11 and 12) between the simulations from the PINN model and the SST k–ω numerical solver is shown to compare the developed model against the best numerical model. The figures show that the developed PINN model is able to give results on par with the SST k–ω model. The RMSE values (comparison with PINN in Table V and VI) substantiate that the performance of the PINN model is similar to the SST k–ω model compared to other numerical models.

In the contour plots (Figs. 5 and 7), the wake created by the turbine is evident from the low velocity (panel a) and low pressure (panel c)

12 September 2026 13:02:35

Published under an exclusive license by AIP Publishing

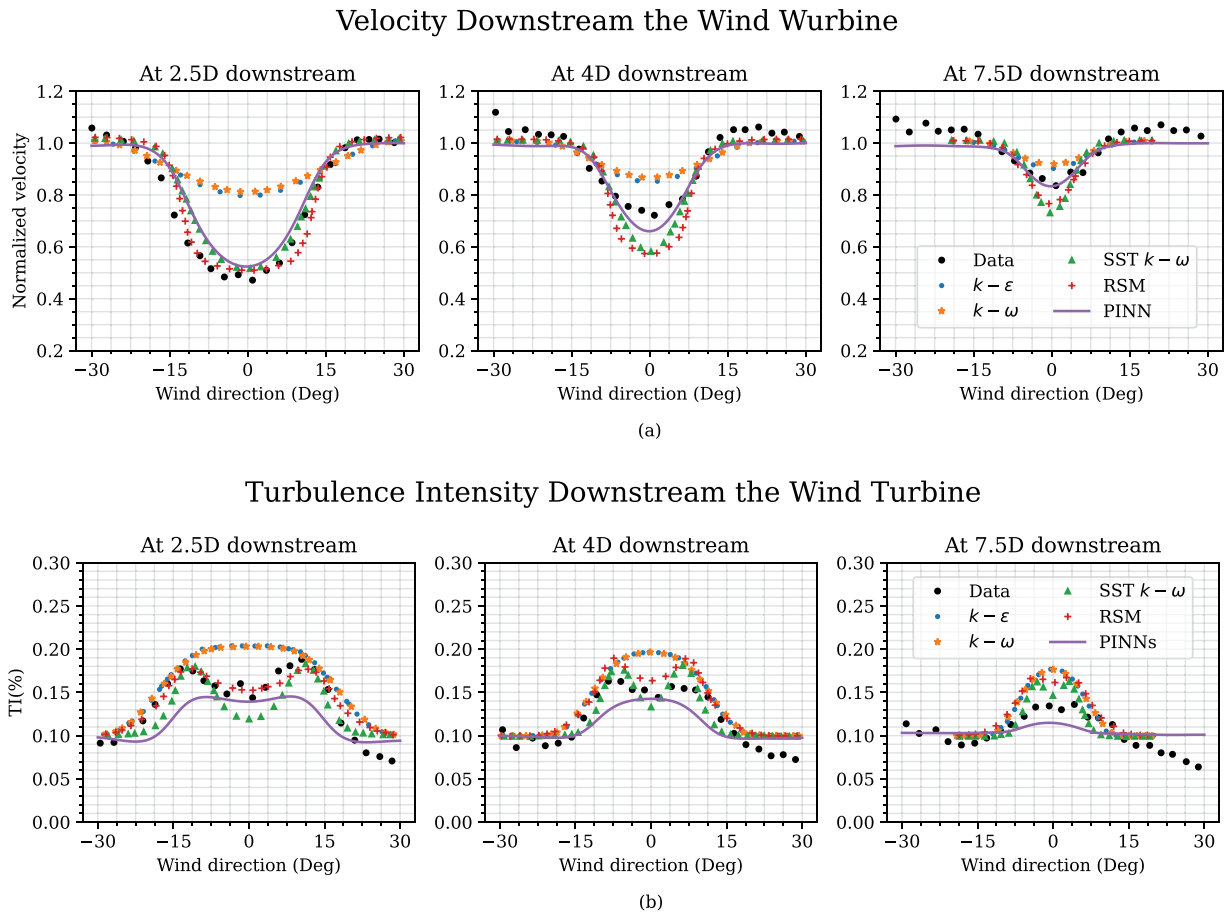


FIG. 8. Comparison of (a) velocity and (b) turbulence intensity predicted by PINN model with data from experiment and numerical solvers at 2.4D, and 7.5D downstream the Nibe-B wind turbine.

regions downstream of the turbine. This causes turbulent transport of momentum from the ambient wind into the wake region as seen in the turbulence intensity contour plot (panel b). The wind turbine generates turbulence in addition to that already produced by the terrain, indicated by the increased turbulence intensity behind the turbine. It is also evident that the turbulence intensity has a sharp structure emerging from the disk edges which is an indication of vortex formation immediately behind the rotor blade tips (marked by numeral I). Further downstream, a shear layer is formed and encloses the wake (marked by numeral II). The far wake region is also well predicted by the PINN model. It is indicated by the expansion of the shear layer to the center by turbulent diffusion (marked by numeral III). The axisymmetric nature of the wake seen

TABLE V. RMSE between the experimental data and the simulations results from different turbulence models and PINN for the Sexbierum HOLEC WPS 30/3 wind turbine. Normalized velocity and TKE are compared at 2.5D, 5.5D, and 8D downstream of the turbine.

		Comparison with exp. data					Comparison with PINN			
Quantiy	Distance	k–ϵ	k–ω	SST k–ω	RSM	PINN	k–ϵ	k–ω	SST k–ω	RSM
Normalized velocity	2.5D	0.1899	0.1975	0.0958	0.0782	0.0768	0.1448	0.1517	0.0232	0.0308
	5.5D	0.0749	0.0808	0.0488	0.0475	0.0433	0.0651	0.0711	0.0138	0.0217
	8D	0.0587	0.0662	0.0522	0.0458	0.0406	0.038	0.0441	0.0140	0.009
Normalized TKE	2.5D	0.0115	0.0118	0.0151	0.0154	0.0170	0.0133	0.0131	0.0032	0.0038
	5.5D	0.0044	0.0044	0.0058	0.0047	0.0062	0.0031	0.0029	0.0022	0.0024
	8D	0.0045	0.0045	0.0039	0.0039	0.0047	0.0024	0.0024	0.0017	0.0021

12 September 2026 13:02:35

Published under an exclusive license by AIP Publishing

TABLE VI. RMSE between the experimental data and the simulations results from different turbulence models and PINN for the Nibe-B wind turbine. Normalized velocity and turbulence intensity are compared at 2.5D, 4D, and 7.5D downstream of the turbine.

		Comparison with exp. data					Comparison with PINN			
Quantiy	Distance	k–ϵ	k–ω	SST k–ω	RSM	PINN	k–ϵ	k–ω	SST k–ω	RSM
Normalized velocity	2.5D	0.1841	0.1925	0.0620	0.0492	0.0666	0.1373	0.1454	0.0190	0.0581
	4D	0.0691	0.0741	0.0629	0.0789	0.0503	0.0773	0.0845	0.0314	0.0529
	7.5D	0.0459	0.0477	0.0509	0.0445	0.0488	0.0253	0.0322	0.0330	0.0245
Turbulence intensity (%)	2.5D	0.0301	0.0298	0.0193	0.0143	0.0277	0.0468	0.0462	0.0201	0.0263
	4D	0.0233	0.0233	0.0139	0.0179	0.0184	0.0329	0.0327	0.0175	0.0279
	7.5D	0.0195	0.0195	0.0130	0.0187	0.0169	0.0260	0.0262	0.0181	0.0253

in the contour plots is a result of the RANS closure being used for the simulations. The RANS turbulence closure fails to simulate highly transient fine vortex tubes, which in turn obstruct the development of complex vortex structures, and the simulated vortex tube moves in a straight line without meandering. The pressure jump across the disk is seen from the pressure contours (marked by numeral IV), which is also an indication of the thrust force exerted by the turbine on the incoming flow. A higher pressure jump indicates a larger thrust force.

Comparison with experimental data and numerical solutions is performed to validate our PINN model at three downstream locations of the wind turbine at a distance of 2.5D, 5.5D, and 8D for the Sexbierum wind turbine. Figure 6(a) shows the normalized wind speed

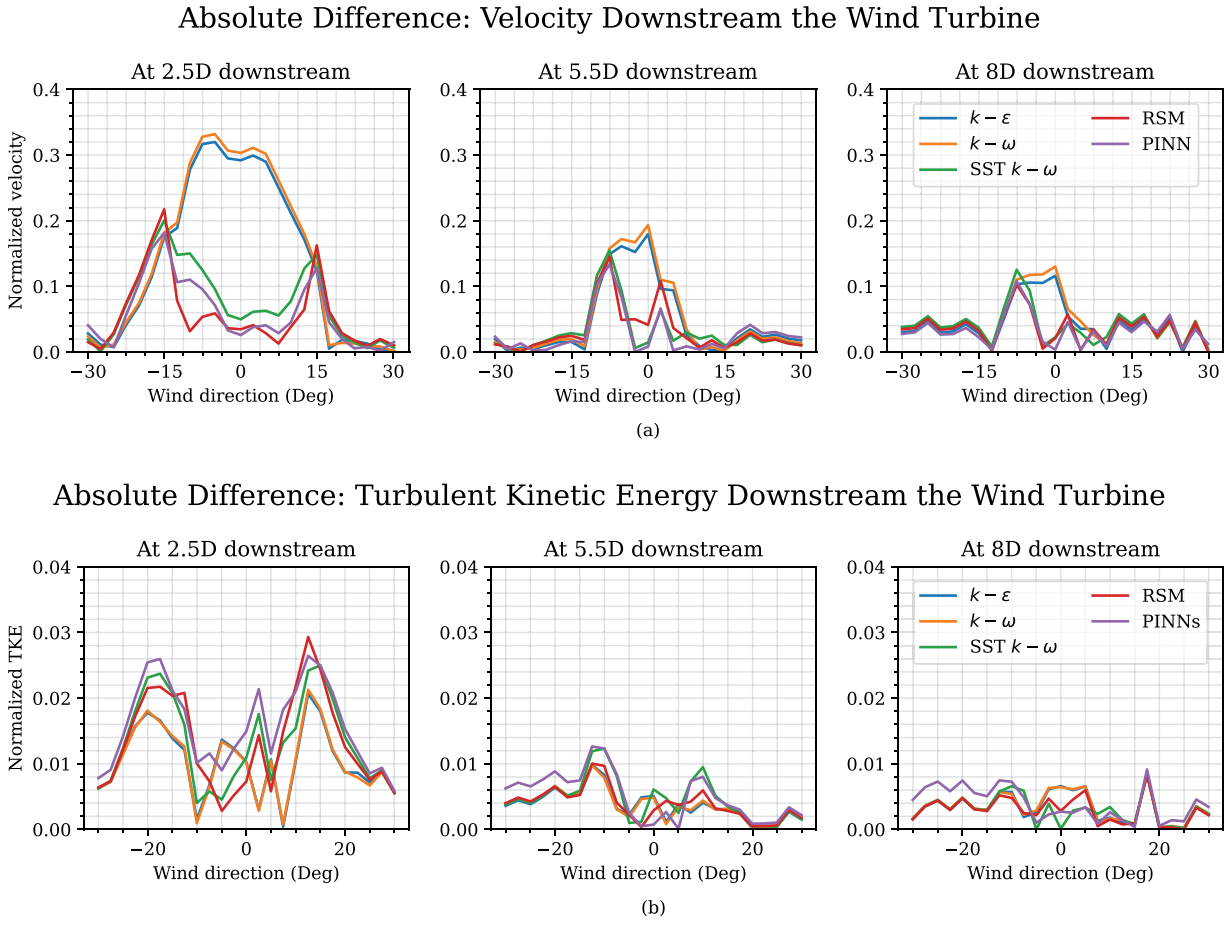


FIG. 9. Absolute difference of (a) velocity and (b) turbulent kinetic energy between the experimental data and the simulations results from different turbulence models and PINN at 2.5D, 5.5D, and 8D downstream the Sexbierum HOLEC WPS 30/3 wind turbine.

12 September 2026 13:02:35

Published under an exclusive license by AIP Publishing

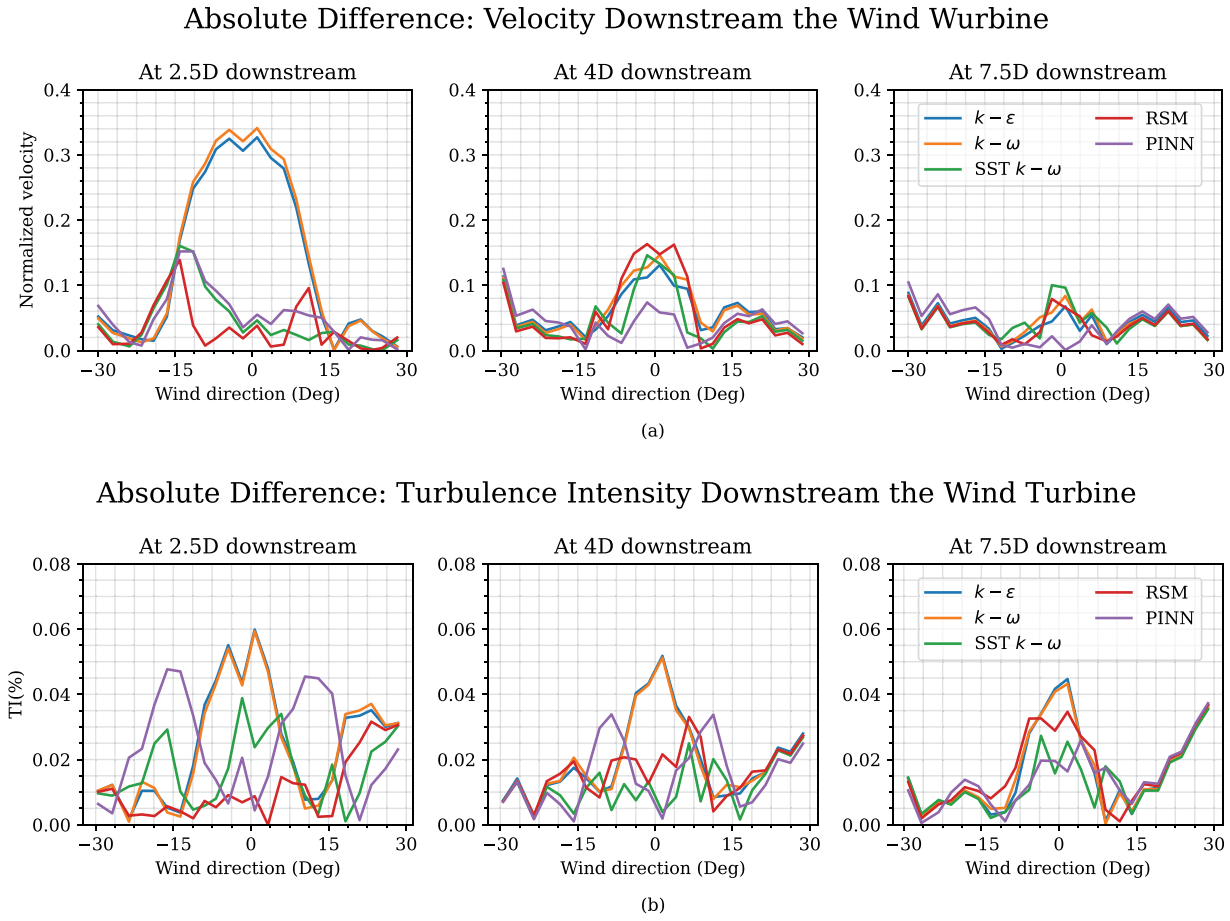


FIG. 10. Absolute difference of (a) velocity and (b) turbulence intensity between the experimental data and the simulations results from different turbulence models and PINN at 2.5D, 4D, and 7.5D downstream the Nibe-B wind turbine.

downstream of the wind turbine as a function of the wind direction. Wind direction indicates the angle between a line passing through the center of the turbine at the hub height, and the line joining the center of the turbine with a line parallel to the turbine, where measurements are taken at the specified downstream distances, as marked in the contour plot [Fig. 5(a)]. The maximum wind speed deficit is observed in the direction behind the center of the turbine (zero degrees) as expected. It can be seen that the wind speed predicted downstream is in good agreement with the experimental data. In addition, the accuracy of the velocity profiles from the PINN model is comparable to the RSM and SST $k-\omega$ models. The RSM and SST $k-\omega$ models are computationally intensive but are more accurate compared to the standard $k-\varepsilon$ and $k-\omega$ models. Figure 6(b) shows the normalized turbulent kinetic energy at three downstream locations. In the near-wake region (2.5D), the PINN model is able to predict the peaks of TKE generated by the tip vortices similar to the SST $k-\omega$ and RSM models. However, the standard $k-\varepsilon$ and $k-\omega$ with baseline coefficients were unable to capture the same. Further downstream (5.5D), the TKE profiles exhibit a more flat peak. Although the TKE is slightly underestimated in the far downstream region (8 D), overall, the PINN models provide simulations that agree well with the experimental data.

Similar results are obtained for the Nibe turbine (Fig. 8). The velocity and turbulence intensity are compared with the experimental data and conventional CFD simulations at locations 2.5D, 4D, and 7.5D downstream of the turbine. Similar to the Sexbierum turbine simulations, here also, the velocity plots [Fig. 8(a)] show that the simulation results agree well with the experiment and the results of the SST $k-\omega$ and RSM models. Figure 8(b) shows that the turbulence intensity forms a plateau downstream of the turbine, which flattens further downstream and follows the trend exhibited by the experimental data. The turbulence intensity is under-predicted far downstream. Although the TKE and turbulence intensity profiles show some deviation from the CFD simulation, the velocity profiles agree well. A major use of the turbulent wake simulations is to estimate the power output from a wind farm where other turbines will be placed in the downstream. Velocity is the most critical input to estimating power [Eq. (5)], and thus, an accurate estimate of the velocity is crucial.

IV. COMPUTATIONAL TIME

For all cases, training is performed on a standalone workstation with two NVIDIA RTX A6000 GPUs.

12 September 2026 13:02:35

Published under an exclusive license by AIP Publishing

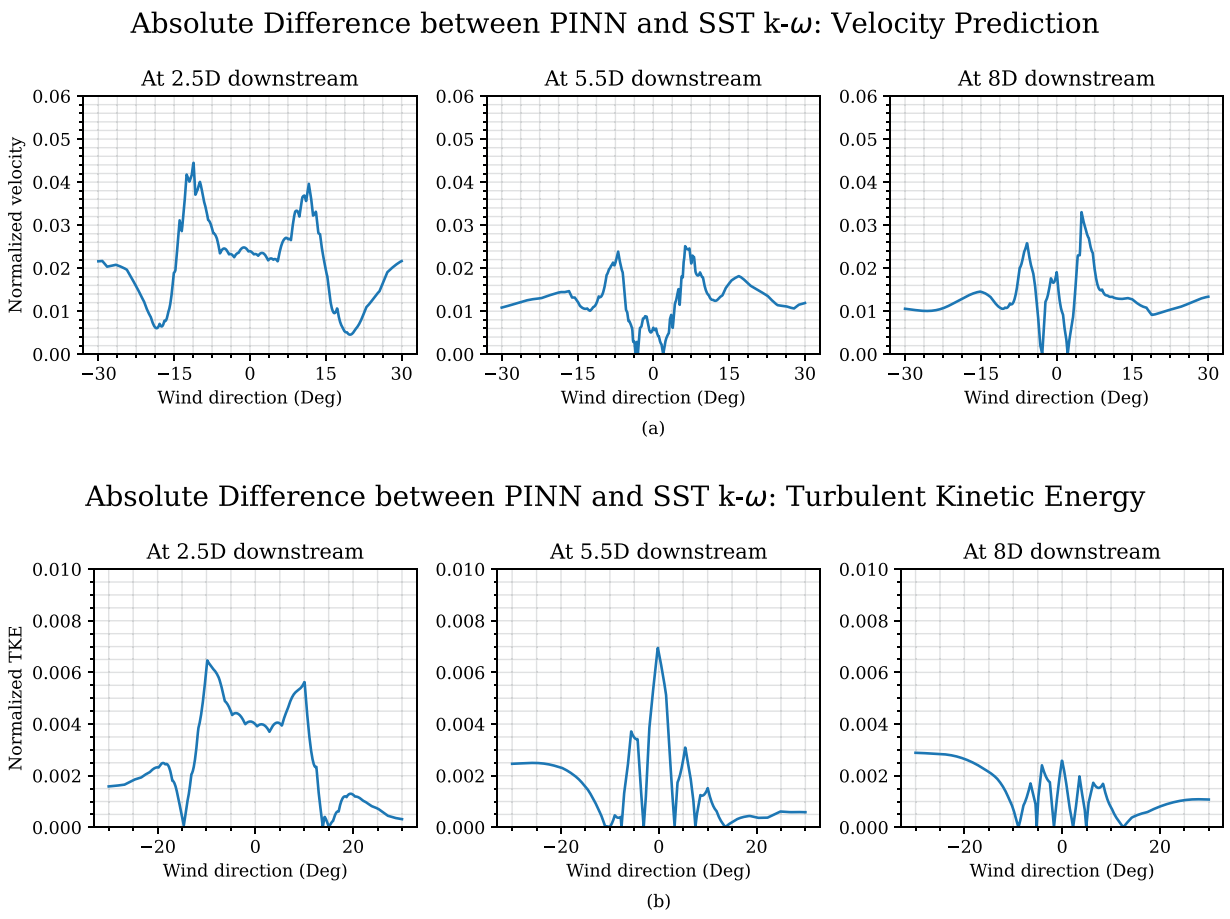


FIG. 11. Absolute difference of (a) velocity and (b) turbulent kinetic energy between the simulations from PINN model and SST k–ω numerical solvers at 2.5D, 5.5D, and 8D downstream the Sexbierum HOLEC WPS 30/3 wind turbine.

A total of 48 000 domain sampling points were used to train the 3D wake behind the wind turbine. The training took 3307.62 min to complete. The inference with the PINN model is much faster and does not require the use of high-end GPUs. In our desktop setup with NVIDIA GeForce RTX 2070, only milliseconds were required to obtain the simulation for the line plots with 1000 spatial points. The inference time for the simulation to plot the contours was less than 10 s using a $500 \times 200 \times 1$ grid. When the model is trained on two GPUs vs one GPU, we found that the speed-up ratio was only 1.4 indicating that a linear scaling does not occur for the present PINN model training. One drawback of all the current PINN frameworks (including our model) is that the training time is high compared to a well-implemented CFD solver; however, with new advances in boundary and initial condition generalization, we hope that the future will see models that are trained once and used for inference in out-of-distribution settings. Another important consideration is that our PINN models are trained only against the PDE residual loss inside the domain. In contrast, data-driven neural operators such as FNO or DeepONets require simulation data from a classical $k - \varepsilon$ RANS CFD solver. Thus, a significant amount of time would be needed to generate the training data itself, which our PINN training avoids. With neural networks, there are opportunities to leverage a transfer learning approach, where the trained model for one type of wind turbine can be used as the starting point for another wind turbine so that training can be completed with less computation as shown next.

A. Transfer learning to reduce training time

As the training time of the PINN model is higher than the runtime of a state-of-the-art numerical solver, we explore the transfer learning strategy to reduce computational training time for new turbulent flow situations for the flow behind a single wind turbine. In transfer learning, a pre-trained model checkpoint is used as the starting point for training models for new tasks/situations. For the 3D wake behind a single wind turbine test case, for a particular turbine, changing the reference velocity (U_{ref}) or turbulence intensity (TI_{ref}) changes the boundary conditions, thrust and power output, thereby requiring new training of the PINN model. We employ transfer learning to show how to reduce this training time.

The experimental setup is as follows: First, a PINN model is trained with $U_{\text{ref}} = 10\,\text{m/s}$ for 25 000 epochs and the converged network's simulation is taken as a reference to evaluate the quality

12 September 2026 13:02:35

Published under an exclusive license by AIP Publishing

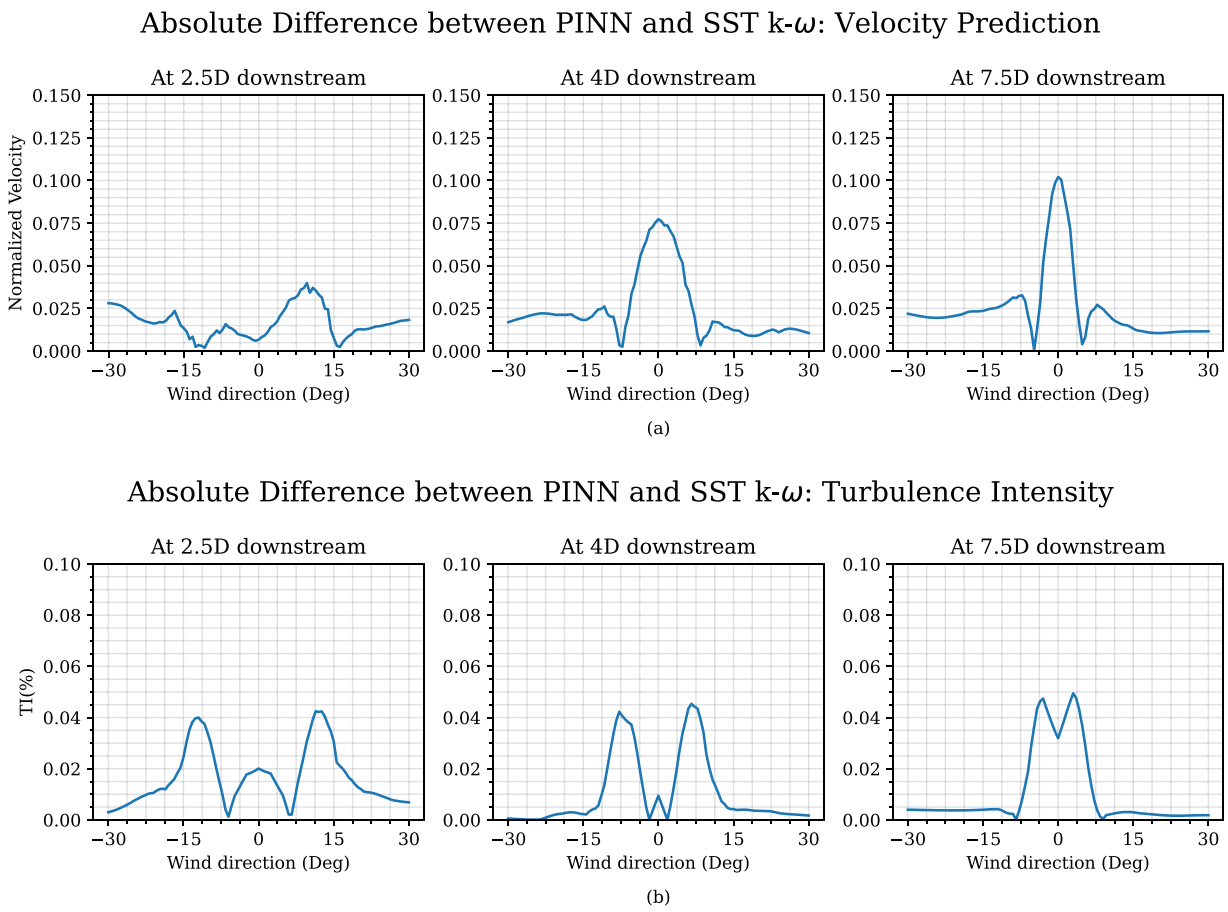


FIG. 12. Absolute difference of (a) velocity and (b) turbulence intensity between the simulations from PINN model and SST k–ω numerical solver at 2.5D, 4D, and 7.5D downstream the Nibe-B wind turbine.

of the simulation produced by transfer learning. Another PINN model with $U_{ref} = 8m/s$ is trained from scratch for 25 000 epochs and is used as the pre-trained model from which transfer learning is applied to simulate the flow with $U_{\text{ref}} = 10\,\text{m/s}$. 5000 epochs of fine-tuning are performed in this transfer learning task. The fine-tuning took 682.32 min compared to the 3307.62 minutes for training from scratch. Figure 13 shows the streamwise velocity and pressure along the centerline at hub height for the model compared

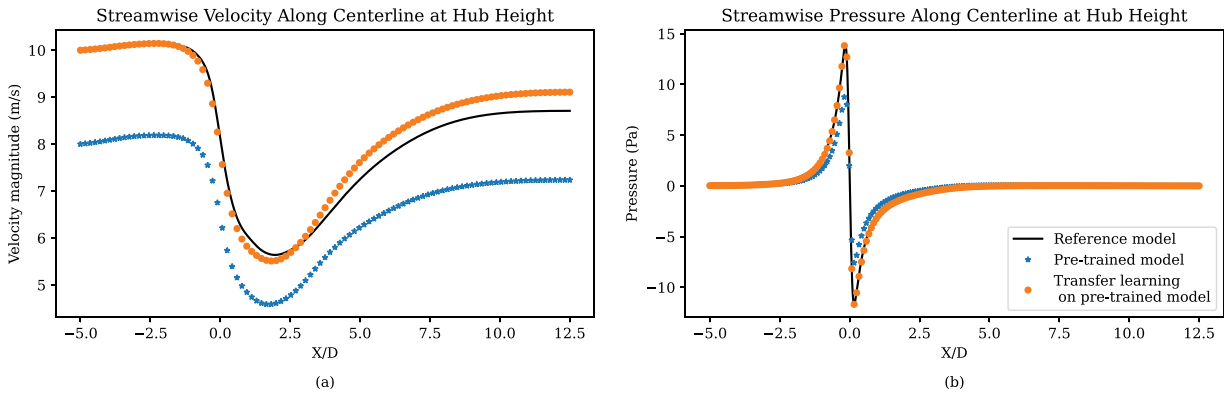


FIG. 13. (a) Streamwise velocity and (b) pressure along the centerline at hub height for the Sexbierum HOLEC WPS 30/3 wind turbine for (i) reference model ($U_{\text{ref}} = 10\,\text{m/s}$), (ii) pre-trained model ($U_{\text{ref}} = 8\,\text{m/s}$), and (iii) transfer learning ($U_{\text{ref}} = 10\,\text{m/s}$).

12 September 2026 13:02:35

Published under an exclusive license by AIP Publishing

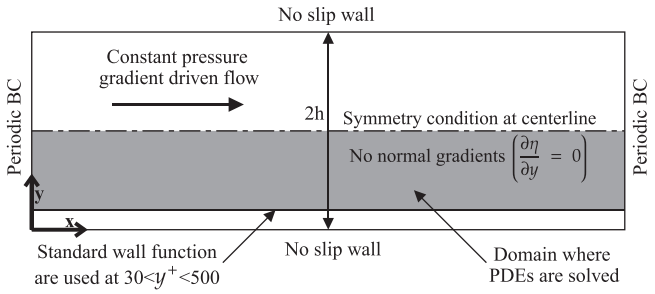


FIG. 14. Schematic of the domain for turbulent channel flow together with the boundary conditions.

to the reference and the pre-trained models. We see that the PINN model with transfer learning converged in one fifth the number of epochs as the model trained from scratch. Transfer learning to speed up training combined with the utility of PINN models to perform fast inference makes them a viable option in wind energy simulators.

V. CONCLUSION

A new physics-informed neural network was developed to simulate turbulent wake using the RANS formulation with the 2-equation $k-\varepsilon$ turbulence closure model. Wall functions and source terms for actuator disk models were also implemented and used. The new PINN model was trained using only PDE residuals and boundary conditions without using data from field experiments or numerical solvers. Hence, the time and expertise the data-driven ML models require to set up and generate/pre-process data from numerical solvers for training are avoided in the developed PINN model. The simulations showed excellent consistency with the references for both standalone wind turbine cases. Although the training time for the PINN models is high compared to the well-implemented parallel numerical solvers, there are quite a few advantages in considering PINNs as a promising approach for solving PDEs. The PINN model provides a mesh-free solution by forming a complex function map between input and output. Hence, during the swift inference, the PINN model can give the solution at any point in the domain out of the box. In addition, by using automatic differentiation, the complex discretization process in conventional numerical solvers is avoided. Further theoretical and implementation optimization, such as boundary and geometry parameterization combined with transfer learning techniques, can enable the trained model to infer new flow cases without the need to train from scratch.

ACKNOWLEDGMENTS

We thank all members of the QUEST Lab at IISc for useful discussions. The authors are grateful for the Prime Minister's Research Fellowship (PMRF) from the Ministry of Education to Azhar Gafoor CTP.

This research was partially supported by research grants from the Ministry of Earth Sciences (MoES/36/OOIS/Extra/84/2022) and Shell India Markets Private Limited.

AUTHOR DECLARATIONS

Conflict of Interest

The authors have no conflicts to disclose.

Author Contributions

Azhar Gafoor CTP: Formal analysis (lead); Investigation (lead); Methodology (equal); Software (equal); Visualization (lead); Writing – original draft (equal). **Sumanth Kumar Boya:** Investigation (supporting); Software (supporting). **Rishi Jinka:** Investigation (supporting); Software (supporting). **Abhineet Gupta:** Conceptualization (equal); Methodology (supporting); Validation (equal). **Ankit Tyagi:** Conceptualization (supporting); Methodology (supporting); Validation (supporting). **Suranjan Sarkar:** Conceptualization (supporting); Methodology (supporting); Validation (supporting). **Deepak Narayanan Subramani:** Conceptualization (equal); Formal analysis (equal); Funding acquisition (lead); Investigation (equal); Methodology (equal); Software (equal); Supervision (lead); Validation (equal); Visualization (equal); Writing – original draft (equal).

DATA AVAILABILITY

The data and codes that support the findings of this study are openly available in Github at https://github.com/quest-lab-iisc/PINN_WakeTurbulenceModel and Zenodo https://doi.org/10.5281/zenodo.13961398, Ref. 76.

APPENDIX A: 2D FLOW CASES

1. Turbulent channel flow (pressure-driven)

The 2D turbulent channel flow is a benchmark test case used to validate new approaches for solving turbulent flow problems. The DNS solutions available for different Reynolds numbers provide excellent data for validating our PINN model and for hyperparameter tuning.

The flow geometry for this test case is set as shown in Fig. 14 (following Ref. 78). The flow between two flat plates separated by a distance of 2 h in the fully developed regime is driven by a constant negative pressure gradient in the streamwise direction, which is balanced by the wall shear stress. The fully developed turbulent channel flow is symmetric with respect to the centerline. Therefore, the simulation is performed only in a domain restricted to half the channel height. The flow is also homogeneous in the streamwise and spanwise directions. Thus, in this work, the simulation is done in 2D. In addition, the ability of the PINN model to develop a homogeneous flow is used as a validation criterion.

The flow is simulated for friction Reynolds numbers (Re_τ) 180, 395, and 590. Re_τ is defined as

$$Re_\tau = \frac{u_\tau h}{\nu}, \tag{A1}$$

where u_τ is the friction velocity, h is half channel width, and ν is the kinematic viscosity. The constant pressure gradient, which drives the flow in the streamwise direction, must be provided. By simplifying the momentum equation in the streamwise direction (x), the pressure gradient can be expressed in terms of density, channel width and friction velocity as

12 September 2026 13:02:35

Published under an exclusive license by AIP Publishing

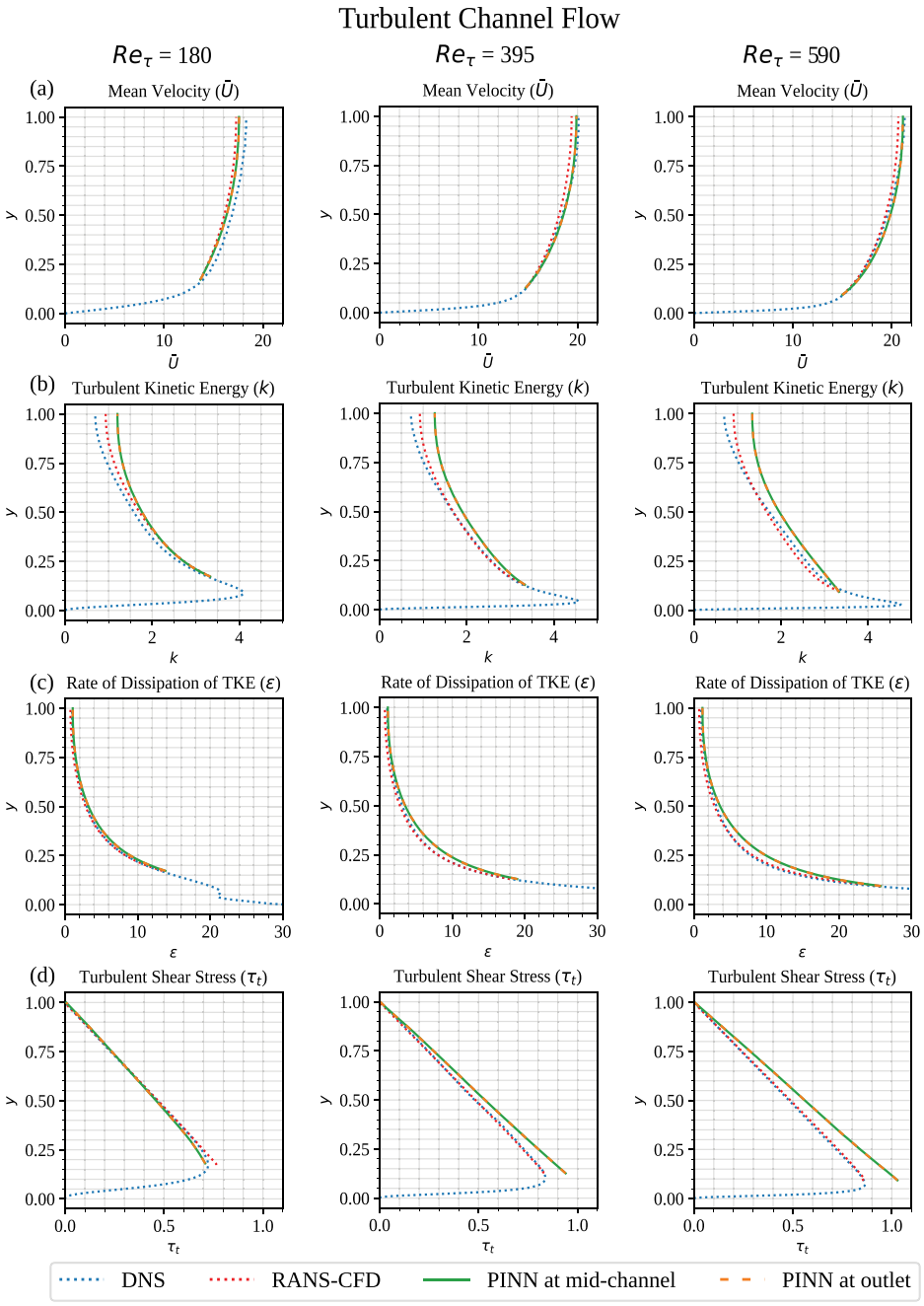


FIG. 15. Comparison of the turbulent channel flow profiles of (a) mean velocity, (b) turbulent kinetic energy, (c) rate of dissipation of TKE, and (d) turbulent shear stress simulated from the PINN model with DNS data and RANS solution from CFD solver for friction Reynolds numbers of 180, 395, and 590.

$$\frac{\partial p}{\partial x} = -\frac{\rho}{h} u_\tau^2. \tag{A2}$$

The inlet and outlet boundaries are given periodic boundary conditions to satisfy homogeneity in the streamwise direction. At the top boundary, symmetry conditions are applied by specifying zero for the velocity in the wall-normal direction and the gradients of other quantities with respect to the wall-normal direction. The velocity, turbulent kinetic energy, and dissipation rate values are specified on the bottom wall using the wall function equations as follows:

$$u = \frac{u_\tau}{\kappa} \ln y^+ + B, \quad k = \frac{u_\tau^2}{\sqrt{C_\mu}}, \quad \varepsilon = \frac{u_\tau^3}{\kappa y}.$$

We used a half channel height of 1 m and a streamwise length of 10 m. As we use wall functions, the bottom boundary is not given at $y = 0m$ but at a specified height $y^+ = 60$ so that the boundary

12 September 2026 13:02:35

Published under an exclusive license by AIP Publishing

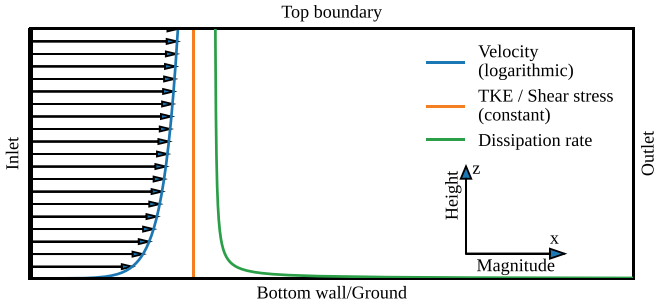


FIG. 16. Illustration of the vertical distribution of horizontal velocity, TKE/shear stress and dissipation rate for the velocity-driven streamwise homogeneous flow. The magnitude of quantities increases from left to right.

falls in the log-law region ($30 < y^+ < 500$) of the turbulent boundary layer. By setting friction velocity (u_τ) equal to 1 m/s, the kinematic viscosity (ν) can be set to 1/180, 1/395, and 1/590 m^2/s using Eq. (A1) to achieve friction Reynolds numbers 180, 395 and 590. Since the channel flow is fully developed, the constant pressure gradient is set equal to -1 Pa/m using Eq. (A2) for density $\rho = 1\,kg/m^3$.

a. PINN model simulations

Figure 15 shows the vertical distribution of mean velocity, turbulent kinetic energy, dissipation rate, and turbulent shear stress at the midpoint and outlet of the channel obtained from our PINN model. Profiles of the DNS and RANS CFD simulations of the fully developed flow are also shown for reference. It can be seen that the simulated profiles match well with the data. Since we employ wall functions to model near-wall behavior, the simulated profiles obtained from our PINN model begin only from the overlap layer of the turbulent boundary layer, as is the case with the RANS CFD simulation. From the figure, it can also be seen that our PINN model well predicts the fully developed or homogeneous condition in the streamwise direction.

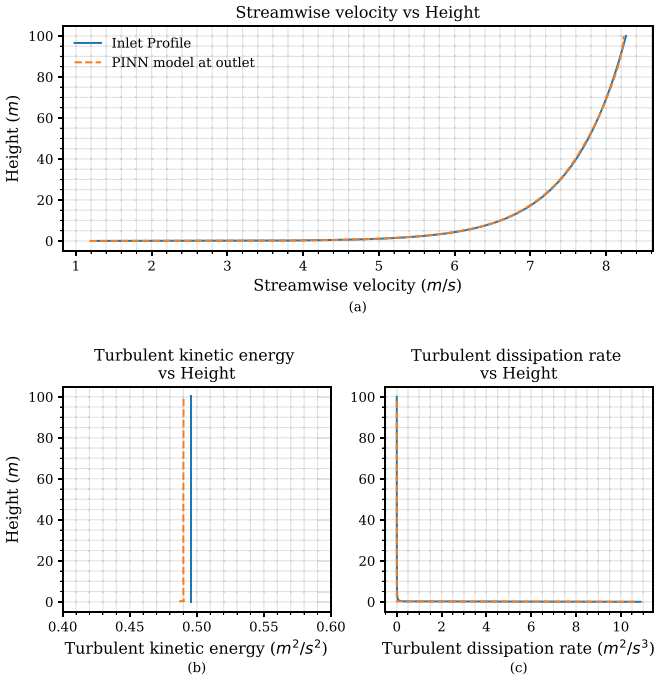
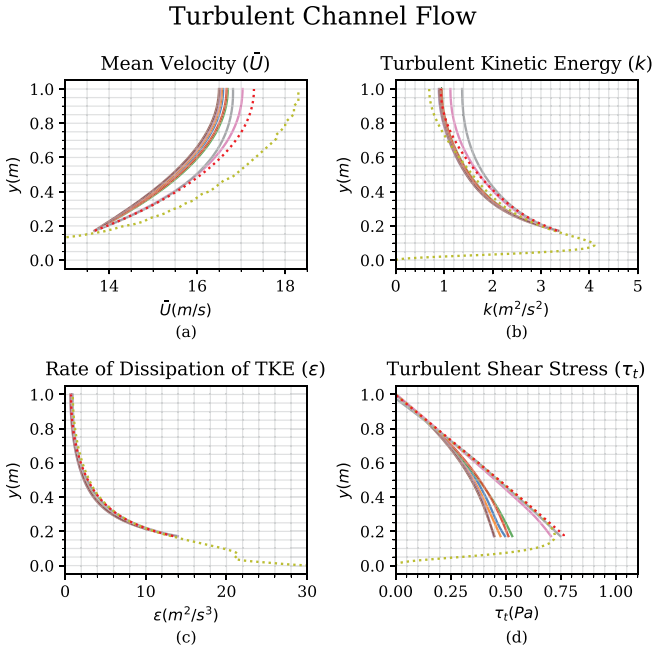


FIG. 17. (a) Streamwise velocity, (b) turbulent kinetic energy, and (c) turbulent dissipation rate profiles of the ABL flow simulated from PINN models are compared to the reference (inlet profile).

2. Velocity-driven streamwise homogeneous flow

As the second test case, we choose the velocity-driven streamwise homogeneous flow with specific inlet conditions. This flow is an added complexity to the turbulent channel flow test case, where the inlet is an atmospheric flow profile[79] commonly used in the atmospheric boundary layer (ABL) flow simulations. We use the standard $k-\varepsilon$ model with the wall functions modified to be consistent with the inlet profiles. Figure 16 shows the inlet flow, turbulent kinetic energy, and dissipation rate profiles in the domain.



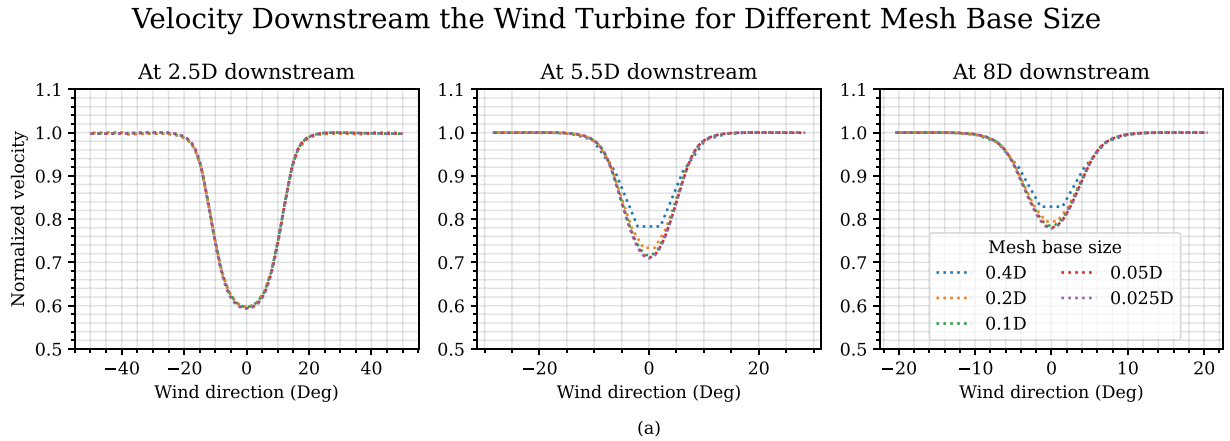
(e) Weights used in PINN models

	α_1	α_2	α_3	α_4	α_b	α_r
PINN 1	1	1	1	1	1	1
PINN 2	10	1	1	1	1	1
PINN 3	10	10	1	1	1	1
PINN 4	10	100	1	1	1	1
PINN 5	100	10	1	1	1	1
PINN 6	100	100	1	1	1	1
PINN 7	10	100	1	1	10	1
PINN 8	10	100	1	1	100	1

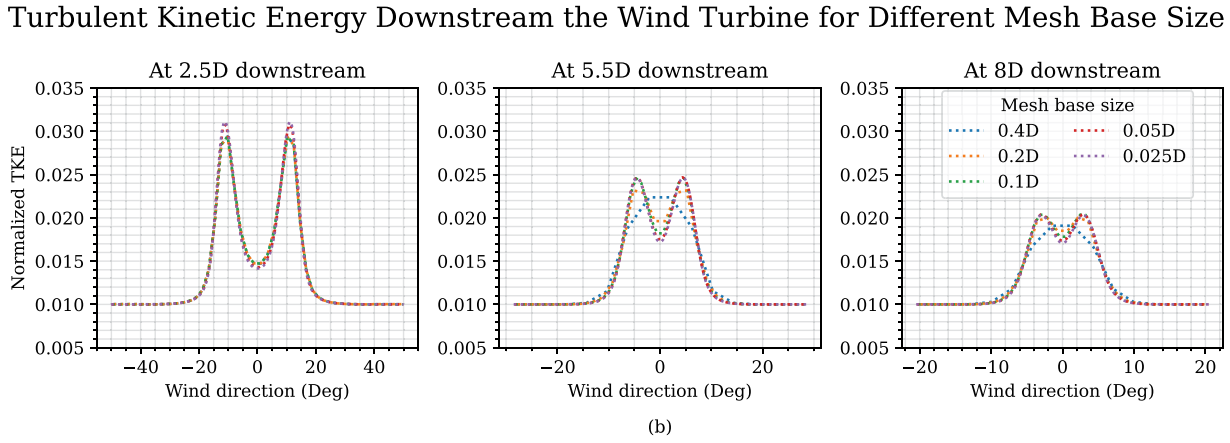
FIG. 18. Comparison of turbulent channel flow profiles of (a) mean velocity, (b) turbulent kinetic energy, (c) rate of dissipation of TKE, and (d) turbulent shear stress simulated from the PINN models, for different weights in Eq. (3), with DNS data and RANS solution from CFD solver for friction Reynolds numbers of 180. Table (e) lists the weights used in PINN models.

12 September 2026 13:02:35

Published under an exclusive license by AIP Publishing



(a)



(b)

FIG. 19. (a) Velocity and (b) turbulent kinetic energy simulations using SST k–ω turbulence model in STARCCM+ numerical solver for different grid resolutions at 2.5D, 5.5D, and 8D downstream the Sexbierum wind turbine. A lower mesh base size denotes a finer grid.

Another motivation to choose this test case is as follows. With the influence of boundaries and no external forcing, the model should be able to simulate the flow, maintaining homogeneity in the streamwise direction for the given inlet conditions. Even with an obstacle (e.g., building or wind turbines) in the domain, sufficiently homogeneous profiles are required upstream and downstream of the obstacle for minimizing the ground/boundary effects.[69] Thus, this test case establishes the ability of our PINN model to simulate homogeneous flow. Specifically, the success criteria for our PINN model's ability to simulate horizontally homogeneous flow are that the simulated downstream velocity, turbulent kinetic energy, and dissipation profiles must match the provided inlet profile.

a. Boundary conditions

For velocity-driven streamwise homogeneous flow, fully developed inlet profiles for mean velocity [Eq. (8a)], turbulent kinetic energy [Eq. (8b)] and turbulent dissipation rate [Eq. (8c)] are specified at the inlet boundary.[79]

At the bottom wall/ground, $z = z_p$ is used in Eq. (8) for specifying conditions. Here, z_p is the height from the ground such that $90 < \frac{z_p u_\tau}{\nu} < 500$.[69] At the top boundary, symmetry boundary conditions are used for velocity, pressure, and other turbulent quantities. A pressure outlet boundary condition is specified for the outlet with no streamwise gradients of other quantities.

b. PINN model simulations

Figure 17 shows the streamwise velocity, turbulent kinetic energy, and dissipation rate profiles obtained from the PINN model plotted against height. The reference inlet profile is also shown for comparison. Here, we use a reference velocity (u_{ref}) of 8 m/s at a reference height (z_{ref}) of 70 m and a turbulence intensity of 7%. The domain has a height of 100 m and a streamwise length of 200 m. The friction velocity, calculated using Eq. (8a) with $z_0 = 0.001$ m, is used to obtain the values of other turbulent quantities. A successful simulation would have obtained horizontally homogeneous flow; that is, the PINN simulations and reference inlet profiles would be identical. The flow is also characterized by constant shear stress, providing a vertical line as the profile of the turbulent kinetic energy (Fig. 17 panel for TKE).

12 September 2026 13:02:35

Published under an exclusive license by AIP Publishing

In Fig. 17, it can be seen that the velocity, TKE, and dissipation rate profiles simulated by the PINN model are in good agreement with the inlet reference profile. Although the deviation seems to be the highest for the TKE profile, the mean absolute error is the least for this variable. The mean absolute errors for velocity is $0.736 \pm 7 \times 10^{-1}$, for TKE is $0.005 \pm 7 \times 10^{-3}$, and for dissipation rate is $0.047 \pm 6 \times 10^{-1}$.

APPENDIX B: WEIGHTS FOR RESIDUAL AND BOUNDARY LOSSES

In conventional numerical methods, in each iteration, the flow parameters are updated in a sequential way. Starting from one of the discretized equations and then approaching the subsequent equations. Hence, iteratively, the convergence is attained by considering the effects of all the equations. Meanwhile, in the training phase of the PINN model, the information from all PDEs is captured simultaneously. Thus, the dominant PDE will influence the parameter learning of the neural network, ignoring the effect of other equations in the domain. To avoid this, a weighting strategy is adopted to give equivalent importance to all the equations and the boundary losses while training. The turbulent channel flow for friction Reynolds numbers of 180 is simulated using the PINN model for weights mentioned in Eq. (3) in a grid search methodology. α_b is the weighting coefficient for boundary losses, α_r is the weighting coefficient for residual losses, and α_{1-4} are the weighting coefficients assigned to the residuals of the governing PDEs: continuity equation, momentum transport equation, TKE transport equation, and TKE dissipation transport equation, respectively. Figure 18 shows the comparison of profiles of the turbulent channel flow simulated from the PINN models with DNS data and RANS solution from CFD solver. The table in the figure lists the weights used for different PINN models. When the weight for the boundary loss (α_b) is given as 1, the model cannot capture the slope of the velocity profile resulting in poor prediction of the turbulent shear stress. On increasing the same to 10, the profiles agree with the RANS simulations, and on further increase, the profiles start to deviate. The weights of "PINN 7" are chosen for the PINN model as the simulation from the model agrees with the numerical results.

APPENDIX C: CFD: MESH CONVERGENCE ANALYSIS

A mesh convergence analysis is done to reduce spatial discretization errors in the CFD simulations and to guarantee a mesh-independent solution. The simulations are based on the single wind turbine of the Sexbierum wind farm. The size of both the background mesh and the refined disk mesh is varied as a function of the turbine diameter for a single turbine by defining a mesh base size. The idea behind the base size is to set other values like surface target size and boundary layer thickness, e.g., proportionally to the base value. The mesh can be refined or coarsened by changing the base size value. All other values defined by the ratio to the base size will change automatically. In the simulations, the base size is decreased progressively by a factor of 2, starting from the coarsest case of 0.4D. Figure 19 shows the velocity and turbulent kinetic energy simulations using STARCCM+ numerical solver at 2.5D, 5.5D, and 8D downstream the single wind turbine of the Sexbierum wind farm for mesh base size from 0.4D to 0.025D. The difference of the calculated velocities and turbulence quantities from simulations for base sizes 0.025D and 0.05D is minimal. Hence, the mesh base size of 0.05D guarantees a mesh-independent solution.

REFERENCES

[1] P. Pandiyan, R. Sitharthan, S. Saravanan, N. Prabaharan, M. R. Tiwari, T. Chinnadurai, T. Yuvaraj, and K. R. Devabalaji, "A comprehensive review of the prospects for rural electrification using stand-alone and hybrid energy technologies," Sustainable Energy Technol. Assess. **52**, 102155 (2022).
[2] Á. López-González, M. Ranaboldo, B. Domenech, and L. Ferrer-Martí, "Evaluation of small wind turbines for rural electrification: Case studies from extreme climatic conditions in Venezuela," Energy **209**, 118450 (2020).
[3] J. Li, P. Liu, and Z. Li, "Optimal design and techno-economic analysis of a solar-wind-biomass off-grid hybrid power system for remote rural electrification: A case study of west China," Energy **208**, 118387 (2020).
[4] P. Moin and K. Mahesh, "Direct numerical simulation: A tool in turbulence research," Annu. Rev. Fluid Mech. **30**, 539–578 (1998).
[5] P. J. Mason, "Large-eddy simulation: A critical review of the technique," Q. J. R. Meteorol. Soc. **120**, 1–26 (1994).
[6] C. G. Speziale, "Analytical methods for the development of Reynolds-Stress closures in turbulence," Annu. Rev. Fluid Mech. **23**, 107–157 (1991).
[7] J. P. Slotnick, A. Khodadoust, J. Alonso, D. Darmofal, W. Gropp, E. Lurie, and D. J. Mavriplis, "CFD vision 2030 study: A path to revolutionary computational aerosciences," Technical Report No. NASA/CR-2014-218178 (NASA, Boeing Research & Technology, Huntington Beach, California, 2014).
[8] C. Pérez, O. Lehmkuhl, J. Jaramillo, G. Colomer, and A. Oliva, "Turbulence modelling and numerical issues: From RANS to DNS and LES," in III Conferência Nacional em Mecânica de Fluidos, Termodinâmica e Energia, Bragança, 2009.
[9] M. Breuer, N. Jovičić, and K. Mazaev, "Comparison of DES, RANS and LES for the separated flow around a flat plate at high incidence," Int. J. Numer. Methods Fluids **41**, 357–388 (2003).
[10] P. M. Doran, "Chapter 7—Fluid flow," in *Bioprocess Engineering Principles*, 2nd ed. (Academic Press, London, 2013), pp. 201–254.
[11] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," J. Comput. Phys. **378**, 686–707 (2019).
[12] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, "Automatic differentiation in machine learning: A survey," J. Mach. Learn. Res. **18**, 1–43 (2018).
[13] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, "Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators," Nat. Mach. Intell. **3**, 218–229 (2021).
[14] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, "Fourier neural operator for parametric partial differential equations," arXiv:2010.08895 (2020).
[15] F. Bartolucci, E. de Bezenac, B. Raonic, R. Molinaro, S. Mishra, and R. Alaifari, "Representation equivalent neural operators: A framework for alias-free operator learning," *Advances in Neural Information Processing Systems* (Curran Associates Inc., Red Hook, NY, USA, 2024), Vol. 36.
[16] A. P. Singh, S. Medida, and K. Duraisamy, "Machine-learning-augmented predictive modeling of turbulent separated flows over airfoils," AIAA J. **55**, 2215–2227 (2017).
[17] K. Duraisamy, G. Iaccarino, and H. Xiao, "Turbulence modeling in the age of data," Annu. Rev. Fluid Mech. **51**, 357–377 (2019).
[18] Z. Zhou, G. He, S. Wang, and G. Jin, "Subgrid-scale model for large-eddy simulation of isotropic turbulent flows using an artificial neural network," Comput. Fluids **195**, 104319 (2019).
[19] R. Maulik, O. San, A. Rasheed, and P. Vedula, "Subgrid modelling for two-dimensional turbulence using neural networks," J. Fluid Mech. **858**, 122–144 (2019).
[20] N. Thuerey, K. Weißenow, L. Prantl, and X. Hu, "Deep learning methods for Reynolds-averaged Navier–Stokes simulations of airfoil flows," AIAA J. **58**, 25–36 (2020).

12 September 2026 13:02:35

Published under an exclusive license by AIP Publishing

[21]Y. Yin, P. Yang, Y. Zhang, H. Chen, and S. Fu, "Feature selection and processing of turbulence modeling based on an artificial neural network," Phys. Fluids **32**, 105117 (2020).
[22]B. Tracey, K. Duraisamy, and J. Alonso, "Application of supervised learning to quantify uncertainties in turbulence and combustion modeling," in 51st AIAA Aerospace Sciences Meeting Including the New Horizons Forum and Aerospace Exposition, AIAA Paper No. AIAA 2013-0259, 2013.
[23]B. D. Tracey, K. Duraisamy, and J. J. Alonso, "A machine learning strategy to assist turbulence model development," in 53rd AIAA Aerospace Sciences Meeting, AIAA Paper No. AIAA 2015-1287, 2015.
[24]J.-X. Wang, J.-L. Wu, and H. Xiao, "Physics-informed machine learning approach for reconstructing Reynolds stress modeling discrepancies based on DNS data," Phys. Rev. Fluids **2**, 034603 (2017).
[25]J.-L. Wu, H. Xiao, and E. Paterson, "Physics-informed machine learning approach for augmenting turbulence models: A comprehensive framework," Phys. Rev. Fluids **3**, 074602 (2018).
[26]M. L. A. Kaandorp and R. P. Dwight, "Data-driven modelling of the Reynolds stress tensor using random forests with invariance," Comput. Fluids **202**, 104497 (2020).
[27]J. Ling, A. Kurzawski, and J. A. Templeton, "Reynolds averaged turbulence modelling using deep neural networks with embedded invariance," J. Fluid Mech. **807**, 155–166 (2016).
[28]L. Zhu, W. Zhang, J. Kou, and Y. Liu, "Machine learning methods for turbulence modeling in subsonic flows around airfoils," Phys. Fluids **31**, 015105 (2019).
[29]X. Sun, W. Cao, Y. Liu, L. Zhu, and W. Zhang, "High Reynolds number airfoil turbulence modeling method based on machine learning technique," Comput. Fluids **236**, 105298 (2022).
[30]M. Milano and P. Koumoutsakos, "Neural network modeling for near wall turbulent flow," J. Comput. Phys. **182**, 1–26 (2002).
[31]H. Xu, W. Zhang, and Y. Wang, "Explore missing flow dynamics by physics-informed deep learning: The parameterized governing systems," Phys. Fluids **33**, 095116 (2021).
[32]J. Wu, H. Xiao, R. Sun, and Q. Wang, "Reynolds-averaged Navier–Stokes equations with explicit data-driven Reynolds stress closure can be ill-conditioned," J. Fluid Mech. **869**, 553–586 (2019).
[33]S. V. Poroseva, J. D. Colmenares F, and S. M. Murman, "On the accuracy of RANS simulations with DNS data," Phys. Fluids **28**, 115102 (2016).
[34]R. L. Thompson, L. E. B. Sampaio, F. A. de Bragança Alves, L. Thais, and G. Mompean, "A methodology to evaluate statistical errors in DNS data of plane channel flows," Comput. Fluids **130**, 1–7 (2016).
[35]R. Fang, D. Sondak, P. Protopapas, and S. Succi, "Deep learning for turbulent channel flow," arXiv:1812.02241 (2018).
[36]J. Bongard and H. Lipson, "Automated reverse engineering of nonlinear dynamical systems," Proc. Natl. Acad. Sci. U. S. A. **104**, 9943–9948 (2007).
[37]M. Schmidt and H. Lipson, "Distilling free-form natural laws from experimental data," Science **324**, 81–85 (2009).
[38]S. L. Brunton, J. L. Proctor, and J. N. Kutz, "Discovering governing equations from data by sparse identification of nonlinear dynamical systems," Proc. Natl. Acad. Sci. U. S. A. **113**, 3932–3937 (2016).
[39]S. H. Rudy, S. L. Brunton, J. L. Proctor, and J. N. Kutz, "Data-driven discovery of partial differential equations," Sci. Adv. **3**, e1602614 (2017).
[40]H. Schaeffer, "Learning partial differential equations via data discovery and sparse optimization," Proc. Math. Phys. Eng. Sci. **473**, 20160446 (2017).
[41]M. Raissi and G. E. Karniadakis, "Hidden physics models: Machine learning of nonlinear partial differential equations," J. Comput. Phys. **357**, 125–141 (2018).
[42]Z. Long, Y. Lu, X. Ma, and B. Dong, "PDE-Net: Learning PDEs from data," in *International Conference on Machine Learning* (PMLR, 2018), pp. 3208–3216.
[43]Z. Long, Y. Lu, and B. Dong, "PDE-Net 2.0: Learning PDEs from data with a numeric-symbolic hybrid deep network," J. Comput. Phys. **399**, 108925 (2019).
[44]H. Eivazi, H. Veisi, M. H. Naderi, and V. Esfahanian, "Deep neural networks for nonlinear model order reduction of unsteady flows," Phys. Fluids **32**, 105104 (2020).
[45]S. Hijazi, G. Stabile, A. Mola, and G. Rozza, "Data-driven POD-Galerkin reduced order model for turbulent flows," J. Comput. Phys. **416**, 109513 (2020).
[46]R. Vinuesa and S. L. Brunton, "Enhancing computational fluid dynamics with machine learning," Nat. Comput. Sci. **2**, 358–366 (2022).
[47]A. Takbiri-Borujeni, H. Kazemi, and N. Nasrabadi, "A data-driven surrogate to image-based flow simulations in porous media," Comput. Fluids **201**, 104475 (2020).
[48]N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, and A. Anandkumar, "Neural operator: Learning maps between function spaces with applications to PDEs," J. Mach. Learn. Res. **24**, 4061–4157 (2023).
[49]Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, "Neural operator: Graph kernel network for partial differential equations," arXiv:2003.03485 (2020).
[50]B. Raonic, R. Molinaro, T. De Ryck, T. Rohner, F. Bartolucci, R. Alaifari, S. Mishra, and E. de Bézenac, "Convolutional neural operators for robust and accurate learning of PDEs," *Advances in Neural Information Processing Systems* (Curran Associates Inc., Red Hook, NY, USA, 2024), Vol. 36.
[51]M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics informed deep learning (Part I): Data-driven solutions of nonlinear partial differential equations," arXiv:1711.10561 (2017).
[52]M. Kazemi, A. Takbiri-Borujeni, S. Takbiri, and A. Kazemi, "Physics-informed data-driven model for fluid flow in porous media," Comput. Fluids **264**, 105960 (2023).
[53]H. Zhang, H. Wang, Z. Xu, Z. Liu, and B. C. Khoo, "A physics-informed neural network-based approach to reconstruct the tornado vortices from limited observed data," J. Wind Eng. Ind. Aerodyn. **241**, 105534 (2023).
[54]J.-D. Müller and P. Cusdin, "On the performance of discrete adjoint CFD codes using automatic differentiation," Int. J. Numer. Methods Fluids **47**, 939–945 (2005).
[55]J. Thomas, E. Dowell, and K. Hall, "Using automatic differentiation to create a nonlinear reduced order model of a computational fluid dynamic solver," in 11th AIAA/ISSMO Multidisciplinary Analysis and Optimization Conference, AIAA Paper No. AIAA 2006-7115, 2006.
[56]C. H. Bischof, H. M. Bücker, A. Rasch, E. Slusanschi, and B. Lang, "Automatic differentiation of the general-purpose computational fluid dynamics package FLUENT," J. Fluids Eng. **129**, 652–658 (2007).
[57]S. Luo, M. Vellakal, S. Koric, V. Kindratenko, and J. Cui, "Parameter identification of RANS turbulence model using physics-embedded neural network," in *International Conference on High Performance Computing* (Springer, 2020), pp. 137–149.
[58]O. Hennigh, S. Narasimhan, M. A. Nabian, A. Subramaniam, K. Tangsali, Z. Fang, M. Rietmann, W. Byeon, and S. Choudhry, "NVIDIA SimNet^{TM}: An AI-accelerated multi-physics simulation framework," in *International Conference on Computational Science* (Springer, 2021), pp. 447–461.
[59]H. Eivazi, M. Tahani, P. Schlatter, and R. Vinuesa, "Physics-informed neural networks for solving Reynolds-averaged Navier–Stokes equations," Phys. Fluids **34**, 075117 (2022).
[60]M.-J. Xiao, T.-C. Yu, Y.-S. Zhang, and H. Yong, "Physics-informed neural networks for the Reynolds-averaged Navier–Stokes modeling of Rayleigh–Taylor turbulent mixing," Comput. Fluids **266**, 106025 (2023).
[61]S. Ghosh, A. Chakraborty, G. O. Brikis, and B. Dey, "RANS-PINN based simulation surrogates for predicting turbulent flows," arXiv:2306.06034 (2023).
[62]V. Kag, K. Seshasayanan, and V. Gopinath, "Physics-informed data based neural networks for two-dimensional turbulence," Phys. Fluids **34**, 055130 (2022).
[63]L. Riccius, A. Agrawal, and P.-S. Koutsourelakis, "Physics-informed tensor basis neural network for turbulence closure modeling," arXiv:2311.14576 (2023).
[64]B. E. Launder and B. I. Sharma, "Application of the energy-dissipation model of turbulence to the calculation of flow near a spinning disc," Lett. Heat Mass Transfer **1**, 131–137 (1974).
[65]A. Crespo, F. Manuel, D. Moreno, E. Fraga, and J. Hernández, "Numerical analysis of wind turbine wakes," in *Proceedings of Workshop on Wind Energy Applications* (Delphi, Greece, 1985), pp. 15–25.
[66]R. Avva, C. Smith, and A. Singhal, "Comparative study of high and low Reynolds number versions of k-epsilon models," in 28th Aerospace Sciences Meeting, AIAA Paper No. AIAA 1990-0246, 1990.
[67]B. E. Launder and D. B. Spalding, "The numerical computation of turbulent flows," Comput. Methods Appl. Mech. Eng. **3**, 269–289 (1974).
[68]G. Kalitzin, G. Medic, G. Iaccarino, and P. Durbin, "Near-wall behavior of RANS turbulence models and implications for wall functions," J. Comput. Phys. **204**, 265–291 (2005).

12 September 2026 13:02:35

Published under an exclusive license by AIP Publishing

[69]B. Blocken, T. Stathopoulos, and J. Carmeliet, "CFD simulation of the atmospheric boundary layer: Wall function problems," Atmos. Environ. **41**, 238–252 (2007).
[70]M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng, see tensorflow.org for "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems" (2015).
[71]J. Rishi, A. Gafoor, S. Kumar, and D. Subramani, "On the training efficiency of shallow architectures for physics informed neural networks," in *International Conference on Computational Science* (Springer, 2024), pp. 363–377.
[72]L. Biewald, see wandb.com. for "Experiment Tracking with Weights and Biases" (2020).
[73]D. V. Dung, N. D. Song, P. S. Palar, and L. R. Zuhal, "On the choice of activation functions in physics-informed neural network for solving incompressible fluid flows," AIAA Paper No. 2023-1803, 2023.
[74]D. P. Kingma, "Adam: A method for stochastic optimization," arXiv:1412.6980 (2014).
[75]X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics* (JMLR Workshop and Conference Proceedings, 2010), pp. 249–256.
[76]A. G. CTP (2024). "quest-lab-iisc/PINN_WakeTurbulenceModel: V 2.0," Zenodo. https://github.com/quest-lab-iisc/PINN_WakeTurbulenceModel
[77]J. Kim, P. Moin, and R. Moser, "Turbulence statistics in fully developed channel flow at low Reynolds number," J. Fluid Mech. **177**, 133–166 (1987).
[78]R. D. Moser, J. Kim, and N. N. Mansour, "Direct numerical simulation of turbulent channel flow up to $Re_\tau = 590$," Phys. Fluids **11**, 943–945 (1999).
[79]P. J. Richards and R. P. Hoxey, "Appropriate boundary conditions for computational wind engineering models using the k-ε turbulence model," J. Wind Eng. Ind. Aerodyn. **46–47**, 145–153 (1993).
[80]R. E. Froude, "On the part played in propulsion by differences of fluid pressure," Trans. Inst. Naval Architects **30**, 390 (1889).
[81]J. N. Sørensen and A. Myken, "Unsteady actuator disc model for horizontal axis wind turbines," J. Wind Eng. Ind. Aerodyn. **39**, 139–149 (1992).
[82]Y.-S. Chen and S.-W. Kim, "Computation of turbulent flow using an extended turbulence closure model," Report No. NASA-CR-179204 (NASA, 1987).
[83]A. El Kasmi and C. Masson, "An extended k–ε model for turbulent flow through horizontal-axis wind turbines," J. Wind Eng. Ind. Aerodyn. **96**, 103–122 (2008).
[84]J. M. Hammersley and D. C. Handscomb, *Monte Carlo Methods* (Methuen & Co. Ltd., London, 1964), Vol. 40, p. 32.
[85]C. Wu, M. Zhu, Q. Tan, Y. Kartha, and L. Lu, "A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks," Comput. Methods Appl. Mech. Eng. **403**, 115671 (2023).
[86]L. Vermeer, J. N. Sørensen, and A. Crespo, "Wind turbine wake aerodynamics," Prog. Aerosp. Sci. **39**, 467–510 (2003).
[87]Y.-T. Wu and F. Porté-Agel, "Atmospheric turbulence effects on wind-turbine wakes: An LES study," Energies **5**, 5340–5362 (2012).
[88]P. K. Jha, E. P. N. Duque, J. L. Bashioum, and S. Schmitz, "Unraveling the mysteries of turbulence transport in a wind farm," Energies **8**, 6468–6496 (2015).
[89]A. Parente, C. Gorlé, J. Van Beeck, and C. Benocci, "Improved k–ε model and wall function formulation for the RANS simulation of ABL flows," J. Wind Eng. Ind. Aerodyn. **99**, 267–278 (2011).
[90]J. W. Cleijne, "Results of sexbierum wind farm: Single wake measurements," Technical Report No. 93–082 (TNO Environmental and Energy Research, Apeldoorn, Netherlands, 1993).
[91]G. J. Taylor, *Wake Measurements on the Nibe Wind-Turbines in Denmark: Data Collection and Analysis,Part 2* (National Power, Technology and Environment Centre, 1990).
[92]Siemens Digital Industries Software, *Simcenter STAR-CCM+ CFD Software* (Siemens 2021).
[93]D. C. Wilcox *et al.*, *Turbulence Modeling for CFD* (DCW Industries, La Canada, CA, 1998), Vol. 2.
[94]F. R. Menter, "Two-equation eddy-viscosity turbulence models for engineering applications," AIAA J. **32**, 1598–1605 (1994).
[95]R. R. Hwang and S.-Y. Jaw, "Second-order closure turbulence models: Their achievements and limitations," Proc. Natl. Sci. Counc., Repub. China, Part A **22**, 703–722 (1998).
[96]B. E. Launder and N. Shima, "Second-moment closure for the near-wall sublayer: Development and application," AIAA J. **27**, 1319–1325 (1989).
[97]E. G. Antonini, D. A. Romero, and C. H. Amon, "Analysis and modifications of turbulence models for wind turbine wake simulations in atmospheric boundary layers," J. Sol. Energy Eng. **140**, 031007 (2018).

12 September 2026 13:02:35

Published under an exclusive license by AIP Publishing